CS6910: Fundamentals of Deep Learning

Lecture 7: Greedy Layerwise Pre-training, Better activation functi initialization, Batch Normalization

Mitesh M. Khapra, Arun Prakash A

AI4Bharat, Department of Computer Science and Engineering, IIT Madras

Module 7.1 : A quick recap of training deep neural n

Mitesh M. Khapra

AI4Bharat, Department of Computer Science and Engineering, IIT Madras

We already saw how to train this network

$\nabla w_t = \frac{\partial \mathscr{L}(w)}{\partial w_t}$

What about a wider network with more inputs:

where, $\nabla w_i = (f(x) - y) * f(x) * (1 - f(x)) *$

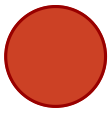
x_i

$w_1 = w_1 - \eta \nabla w_1$

y

x

σ

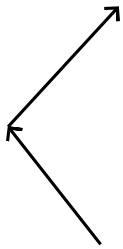


w

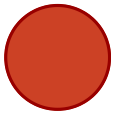
$w_{t+1} = w_t - \eta \nabla w_t$ where,

$= (f(x) - y) * f(x) * (1 - f(x)) * x$

y



σ



x_1

x_3

x_2

w_1

w_3

w_2

$w_2 = w_2 - \eta \nabla w_2$

$w_3 = w_3 - \eta \nabla w_3$

What if we have a deeper network ?

We can now calculate ∇w_1 using chain rule:

$$\frac{\partial \mathscr{L}(w)}{\partial w_1} = \frac{\partial \mathscr{L}(w)}{\partial y} \cdot \frac{\partial y}{\partial a_3} \cdot \frac{\partial a_3}{\partial h_2} \cdot \frac{\partial h_2}{\partial a_2} \cdot \frac{\partial a_2}{\partial h_1} \cdot \frac{\partial h_1}{\partial a_1} \cdot \frac{\partial a_1}{\partial w_1}$$

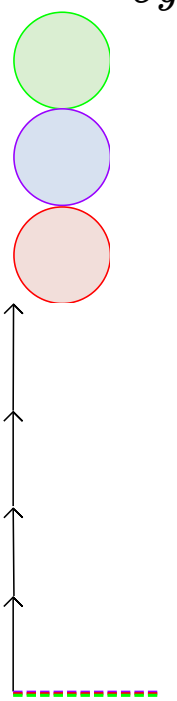
$a_i = w_i h_{i-1}; h_i = \sigma(a_i)$

$a_1 = w_1 * x = w_1 * h_0$

$*$ ..... $*$

h_0

$= \frac{\partial \mathscr{L}(w)}{\partial y}$



y

h_2

a_3

w_3

a_2

w_2

h_1

a_1

w_1

σ

σ

σ

$x = h_0$

In general,

$$\nabla w_i = \frac{\partial \mathcal{L}(w)}{\partial y}$$

$* \ldots\ldots\ldots\ldots * h_i - 1$

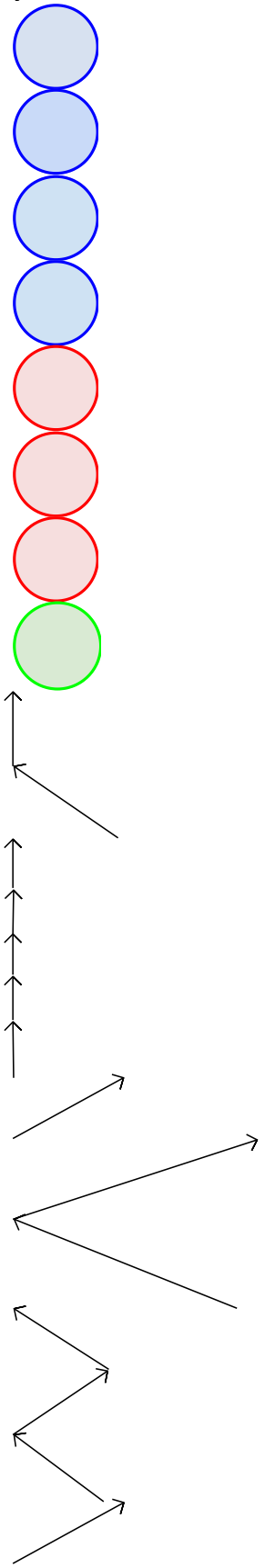
Notice that ∇w_i is proportional to the corresponding input $h_i - 1$

(we will use this fact later)

What happens if we have a network which is deep and wide?

How do you calculate ∇w_2=?

It will be given by chain rule applied across multiple paths



σ

w_1

w_2

w_3

x_1

x_2

x_3

y

σ

σ

σ

σ

σ

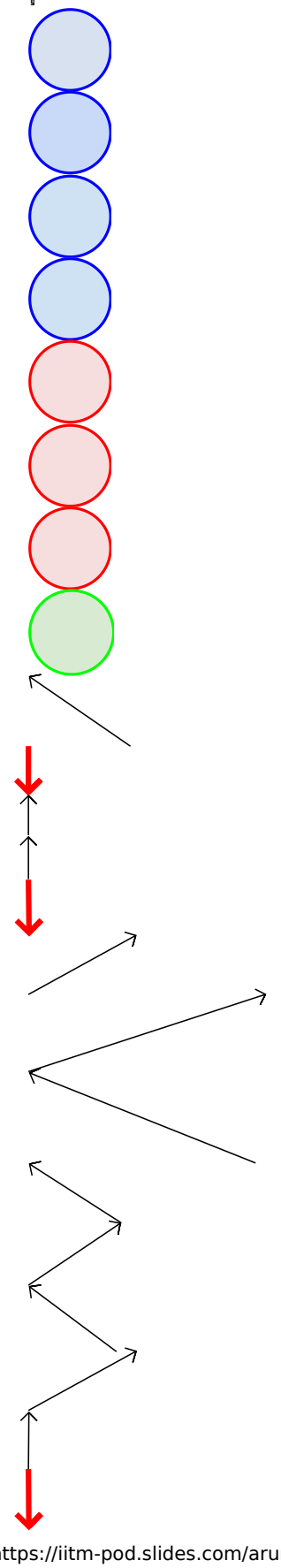
σ

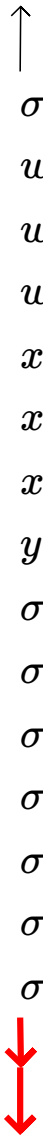
σ

What happens if we have a network which is deep and wide?

How do you calculate ∇w_2=?

It will be given by chain rule applied across multiple paths

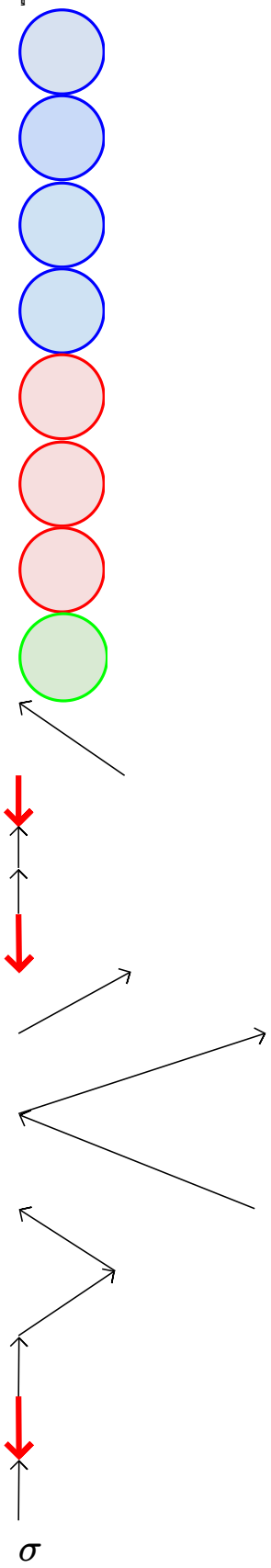



σ
w_1
w_2
w_3
x_1
x_2
x_3
y
σ
σ
σ
σ
σ
σ
σ

What happens if we have a network which is deep and wide?

How do you calculate ∇w_2=?

It will be given by chain rule applied across multiple paths



w_1

w_2

w_3

x_1

x_2

x_3

y

σ

σ

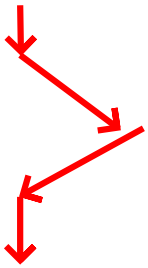
σ

σ

σ

σ

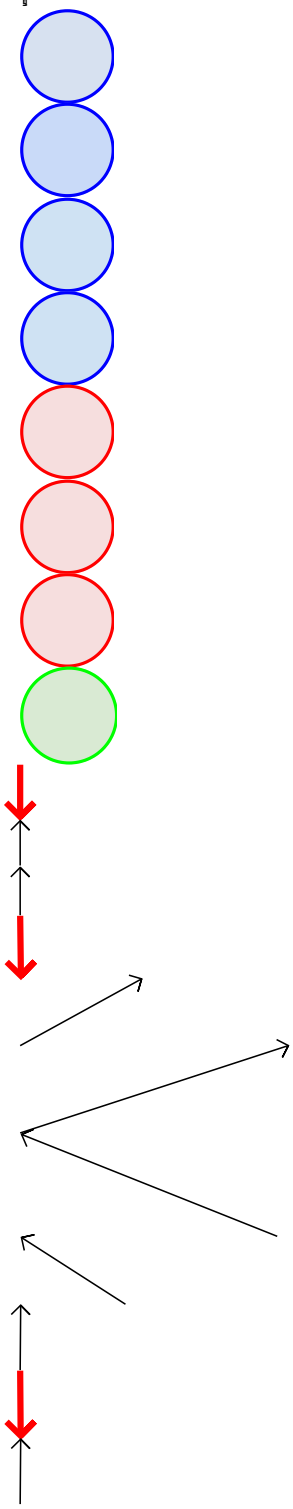
σ



What happens if we have a network which is deep and wide?

How do you calculate ∇w_2=?

It will be given by chain rule applied across multiple paths



σ

w_1

w_2

w_3

x_1

x_2

x_3

y

σ

σ

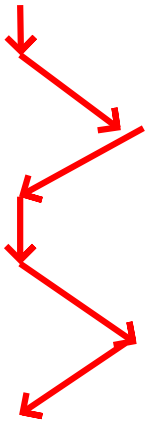
σ

σ

σ

σ

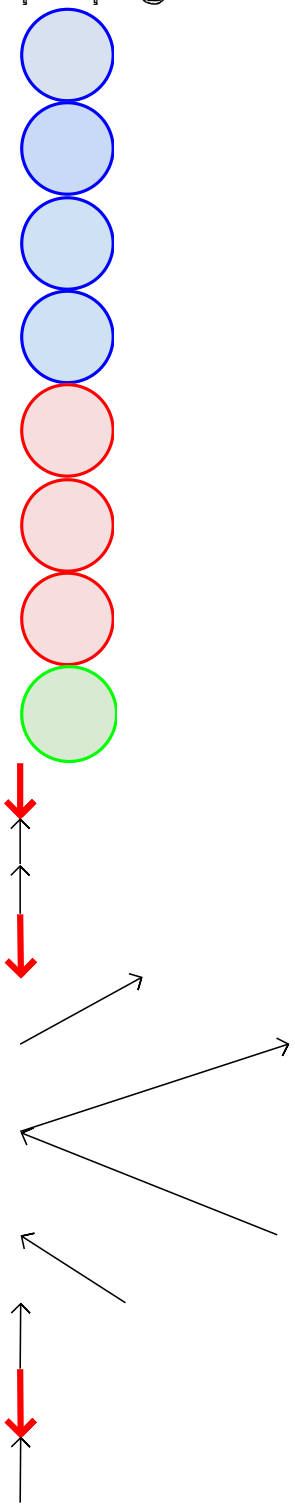
σ



What happens if we have a network which is deep and wide?

How do you calculate ∇w_2=?

It will be given by chain rule applied across multiple paths (We saw this in detail when we studied back propagation)



w_2

w_3

x_1

x_2

x_3

y

σ

σ

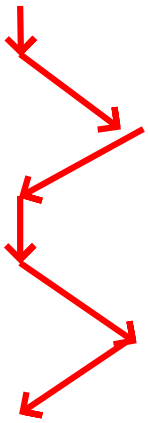
σ

σ

σ

σ

σ



Training Neural Networks is a *Game of Gradients* (played using any of the existing gradient based approaches that we discussed)

Things to remember

The gradient tells us the responsibility of a parameter towards the loss

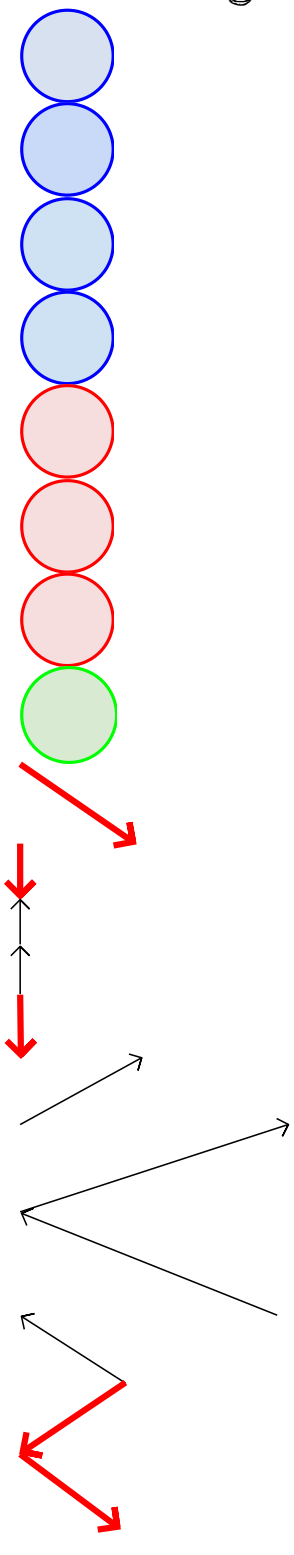
The gradient w.r.t. a parameter is proportional to the input to the parameters (recall the "..... $*$ x" term or the "..... $* h_{i-1}$" term in the formula for ∇w_i)

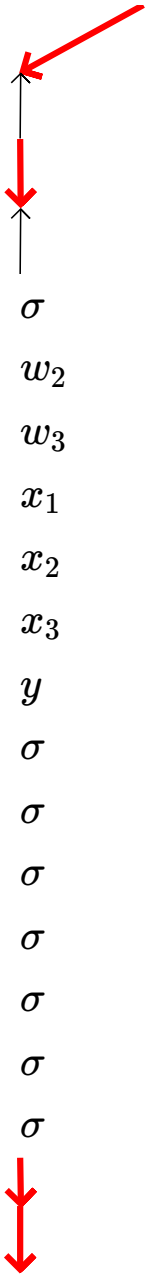
Backpropagation was made popular by Rumelhart et.al in 1986

However when used for really deep networks it was not very successful

In fact, till 2006 it was very hard to train very deep networks

Typically, even after a large number of epochs the training did not converge





σ

w_2

w_3

x_1

x_2

x_3

y

σ

σ

σ

σ

σ

σ

σ

Learning representations by back-propagating errors

David E. Rumelhart*, Geoffrey E. Hinton† & Ronald J. Williams*

* Institute for Cognitive Science, C-015, University of California, San Diego, La Jolla, California 92093, USA
† Department of Computer Science, Carnegie-Mellon University, Pittsburgh, Philadelphia 15213, USA

We describe a new learning procedure, back-propagation, for networks of neurone-like units. The procedure repeatedly adjusts the weights of the connections in the network so as to minimize a measure of the difference between the actual output vector of the net and the desired output vector. As a result of the weight adjustments, internal 'hidden' units which are not part of the input or output come to represent important features of the task domain, and the regularities in the task are captured by the interactions of these units. The ability to create useful new features distinguishes back-propagation from earlier, simpler methods such as the perceptron-convergence procedure[1].

Module 7.2 : Unsupervised pre-training

Mitesh M. Khapra

AI4Bharat, Department of Computer Science and Engineering, IIT Madras

What has changed now? How did Deep Learning become so popular despite this problem with training large networks?

1 G. E. Hinton and R. R. Salakhutdinov. Reducing the dimensionality of data with 313(5786):504–507, July 2006.

What has changed now? How did Deep Learning become so popular despite this problem with training large networks?

Well, until 2006 it wasn't so popular

1 G. E. Hinton and R. R. Salakhutdinov. Reducing the dimensionality of data with 313(5786):504–507, July 2006.

What has changed now? How did Deep Learning become so popular despite this problem with training large networks?

Well, until 2006 it wasn't so popular

The field got revived after the seminal work of Hinton and Salakhutdi in 2006

1 G. E. Hinton and R. R. Salakhutdinov. Reducing the dimensionality of data wit 313(5786):504–507, July 2006.

Let's look at the idea of unsupervised pre-training introduc paper ...

Let's look at the idea of unsupervised pre-training introduced in this pa
(note that in this paper they introduced the idea in the context of RBMs
we will discuss it in the context of Autoencoders)

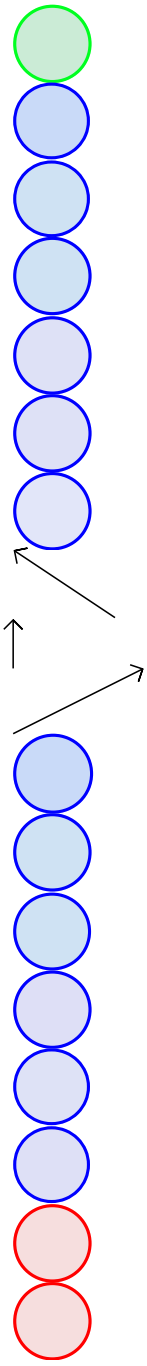
Consider the deep neural network shown in this figure

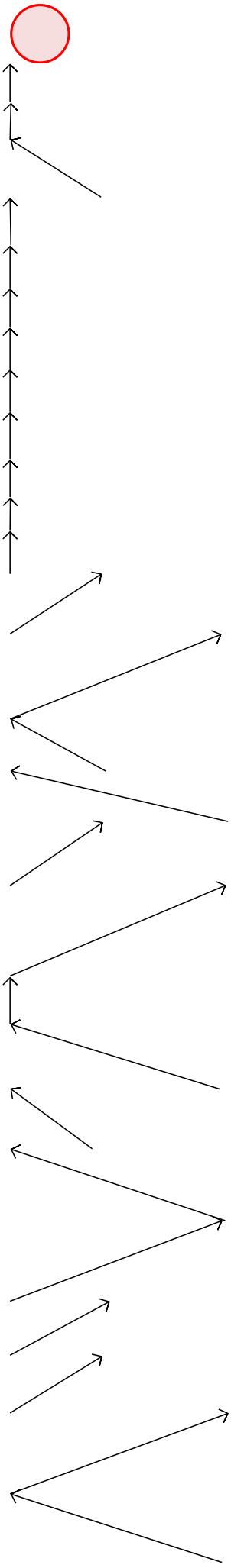
Let us focus on the first two layers of the network x and h_1

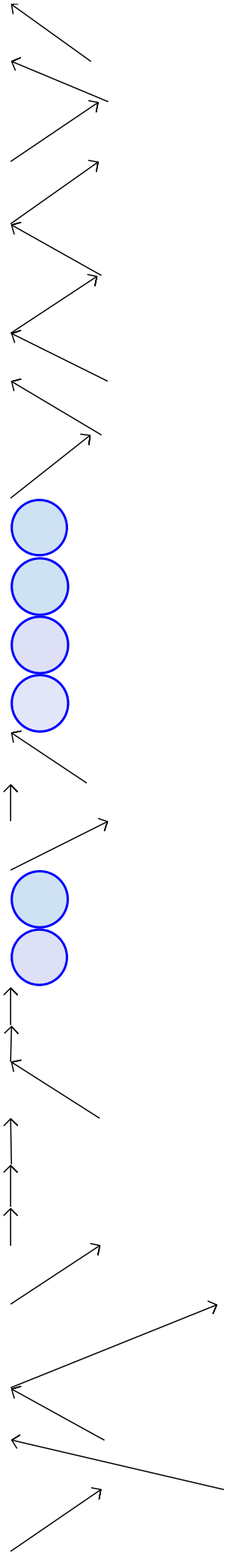
We will first train the weights between these two layers using an **unsupervised objective**

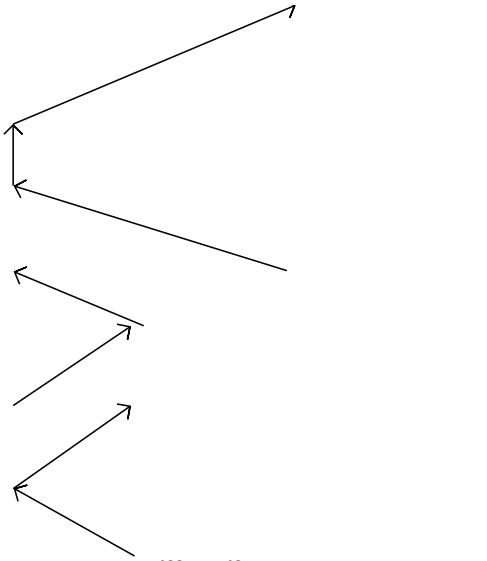
Note that we are trying to reconstruct the input (x) from the hidden representation (h_1)

We refer to this as an unsupervised objective because it does not involve the output label (y) and only uses the input data (x)

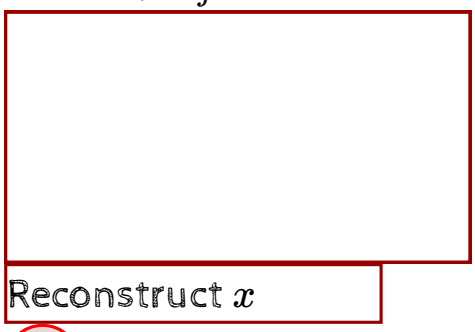


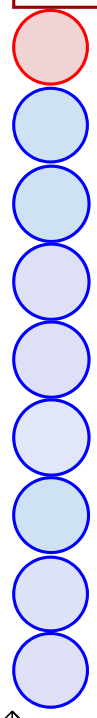


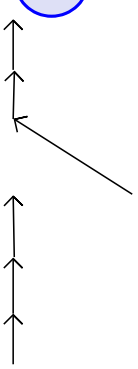


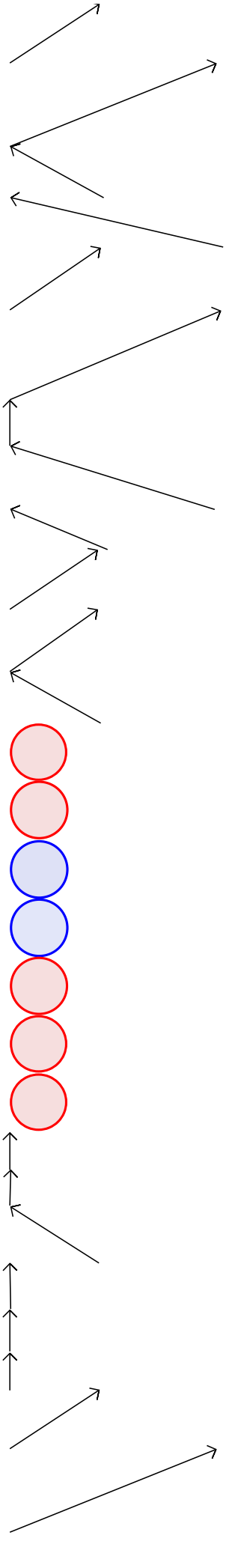


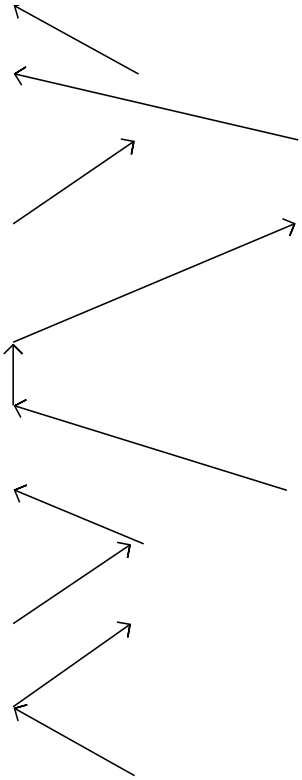
$$min \frac{1}{m} \sum_{i=1}^{m} \sum_{j=1}^{n} (\hat{x}_{ij} - x_{ij})^2$$









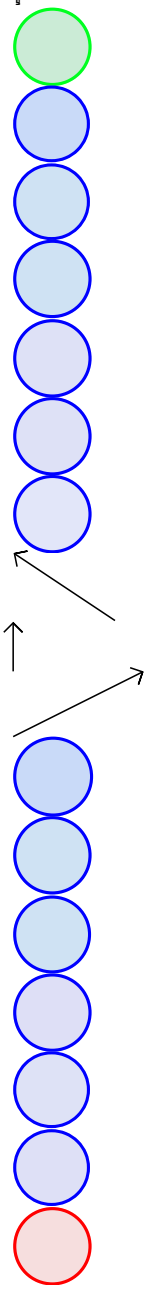


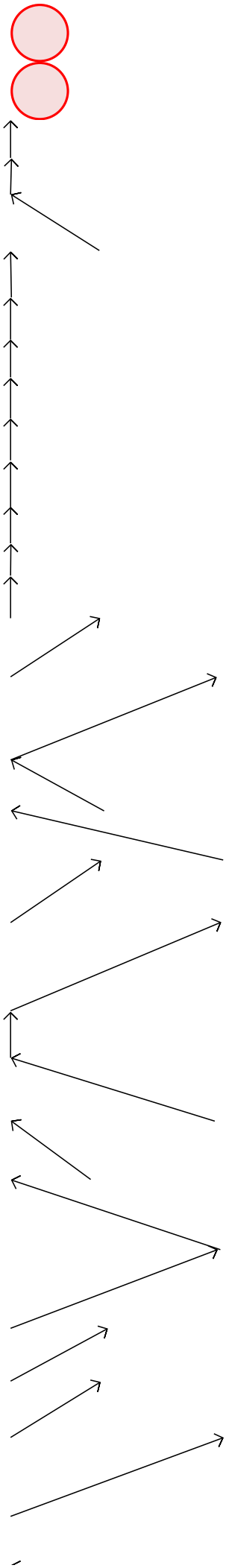
At the end of this step, the weights in layer 1 are trained such that h_1 captures an abstract representation of the input x

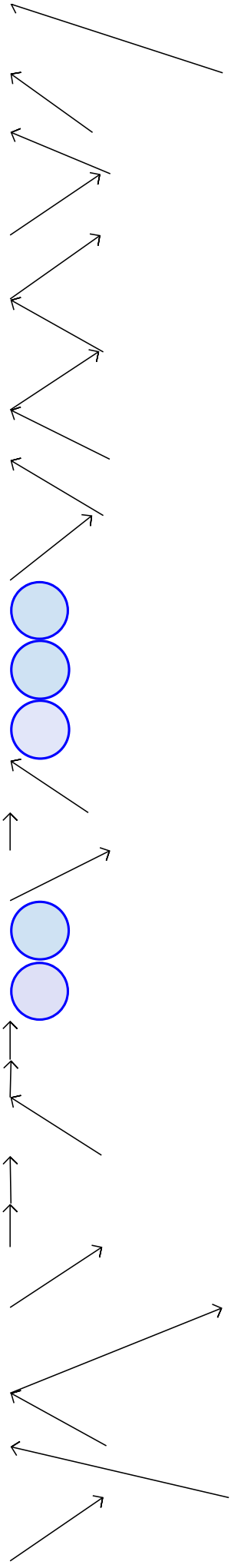
We now fix the weights in layer 1 and repeat the same process with layer 2

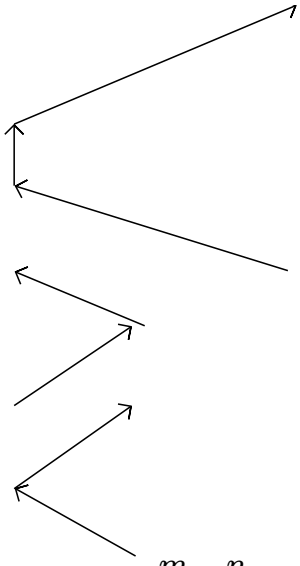
At the end of this step, the weights in layer 2 are trained such that captures an abstract representation of h_1

We continue this process till the last hidden layer (i.e., the layer before the output layer) so that each successive layer captures an abstract representation of the previous layer

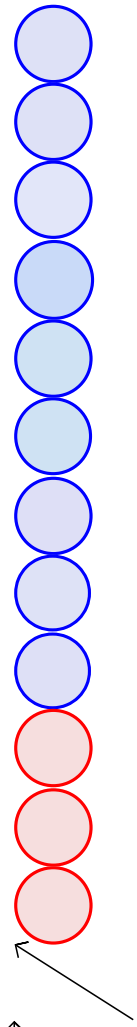


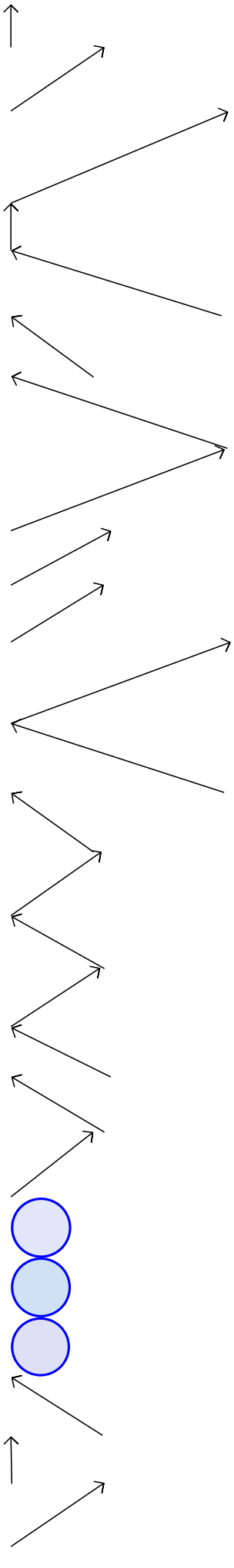


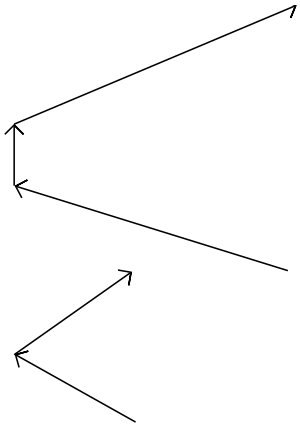




$$min \frac{1}{m} \sum_{i=1}^{m} \sum_{j=1}^{n} (\hat{h}_{1_{ij}} - h_{1_{ij}})^2$$







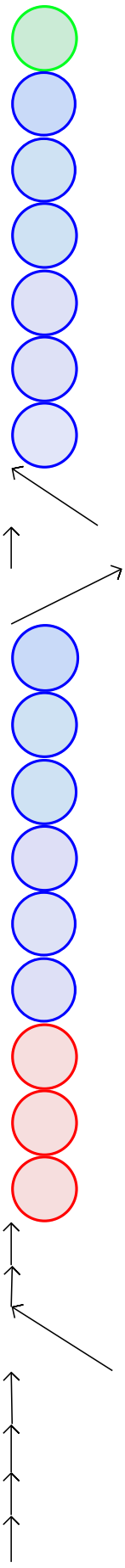
x

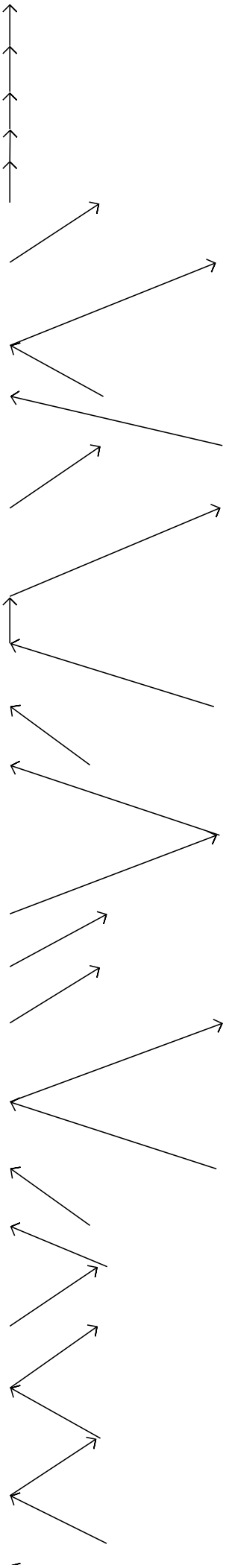
h_1

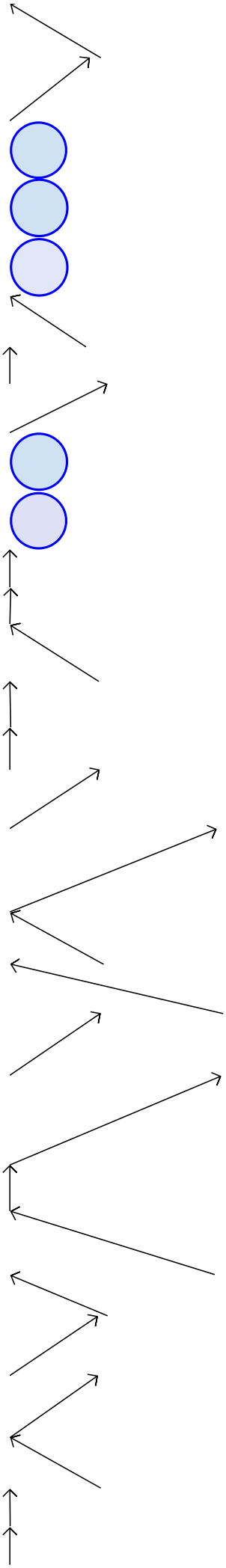
h_2

$\hat{h_1}$

After this layerwise pre-training, we add the output layer and train the whole network using the task specific objective







↑

x_1

x_2

x_3

$\arg\min \frac{1}{m} \sum_{i=1}^{m} (\hat{y}_i - f(x_i))^2$

Note that, in effect we have initialized the weights of the network using the greedy unsupervised objective and are now fine tuning these weights using the supervised objective

Why does this work better?

1 The difficulty of training deep architectures and effect of unsupervised pre-training - Erhan et a

2 Exploring Strategies for Training Deep Neural Networks, Larocelle et al,2009

Is it because of better optimization?

Is it because of better regularization?

Let's see what these two questions mean and try to answer them based
some (among many) existing studies [1,2]

Why does this work better?

Is it because of better optimization?

Is it because of better regularization?

What is the optimization problem that we are trying to solve?

$$\text{minimize } \mathscr{L}(\theta) = \frac{1}{m}\sum_{i=1}^{m}(y_i - f(x_i))^2$$

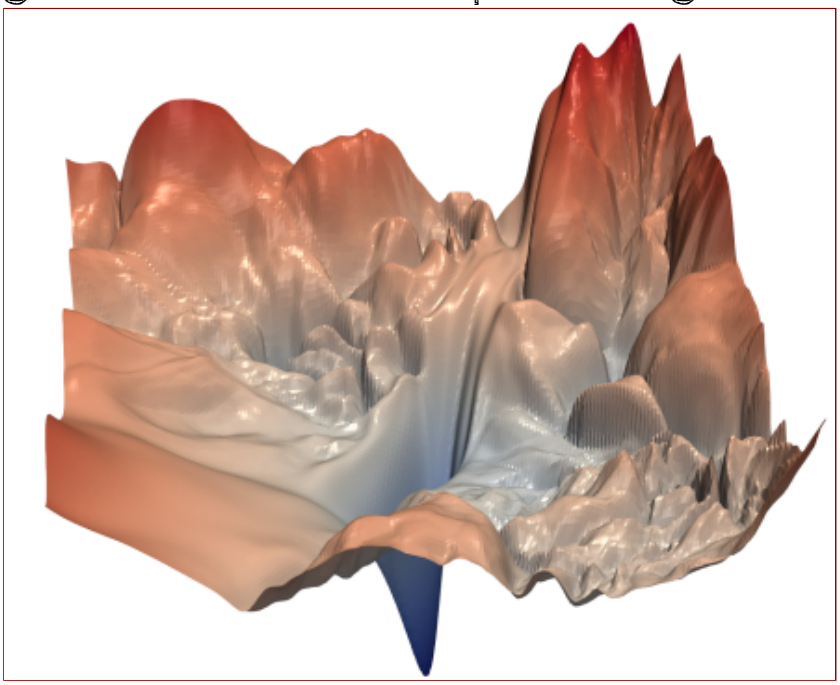
Is it the case that in the absence of unsupervised pre-training we are nc
drive $(\mathscr{L}(\theta))$ to 0 even for the training data (hence poor optimization)
Let us see this in more detail ...

The error surface of the supervised objective of a Deep Neural Network is highly non-convex

With many hills and plateaus and valleys

Given that large capacity of DNNs it is still easy to land in one of these 0 error regions

Indeed Larochelle et.al.1 show that if the last layer has large capacity then $\mathscr{L}(\theta)$ goes to 0 even without pretraining



However, if the capacity of the network is small, unsupervised pretraining helps

Why does this work better?

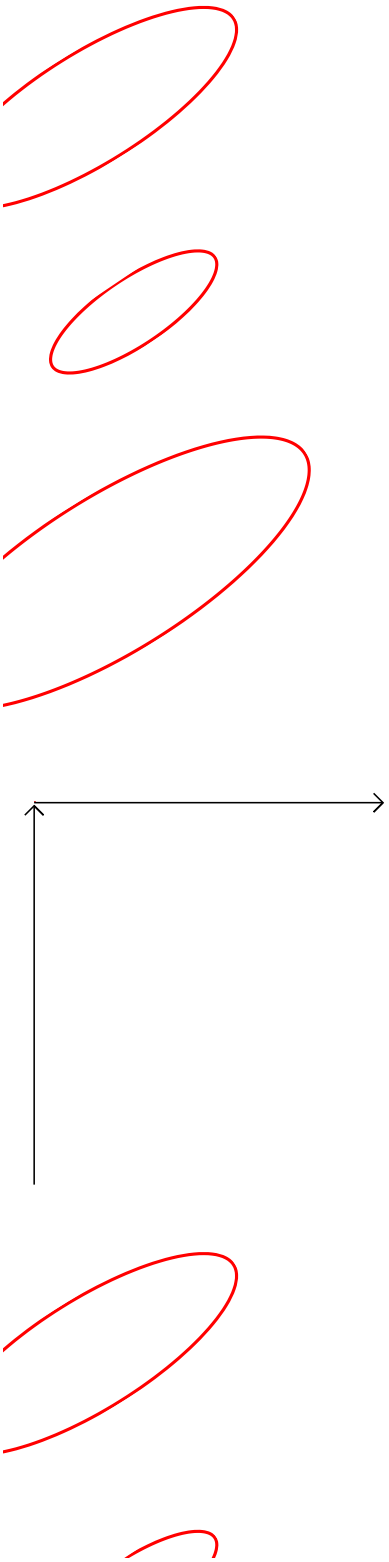
Is it because of better optimization?

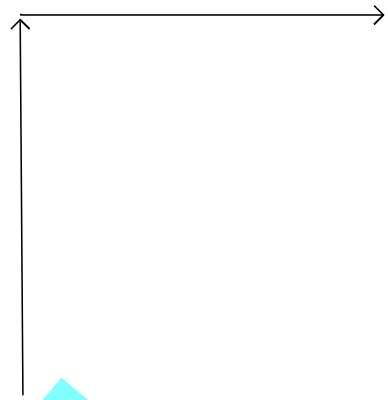
Is it because of better regularization?

What does regularization do? It constrains the weights to certain regions of the parameter space

L-1 regularization: constrains most weights to be 0

L-2 regularization: prevents most weights from taking large values





L_1

L_2

w_1

w_2

w_1

w_2

w^*

w^*

Indeed, pre-training constrains the weights to lie in only certain regions of the parameter space

Specifically, it constrains the weights to lie in regions where the characteristics of the data are captured well (as governed by the unsupervised objective)

$$\Omega(\theta) = \frac{1}{m}\sum_{i=1}^{m}\sum_{j=1}^{n}(x_{ij} - \hat{x}_{ij})^2$$

Unsupervised objective:

We can think of this unsupervised objective as an additional constraint on the optimization problem

Supervised objective:

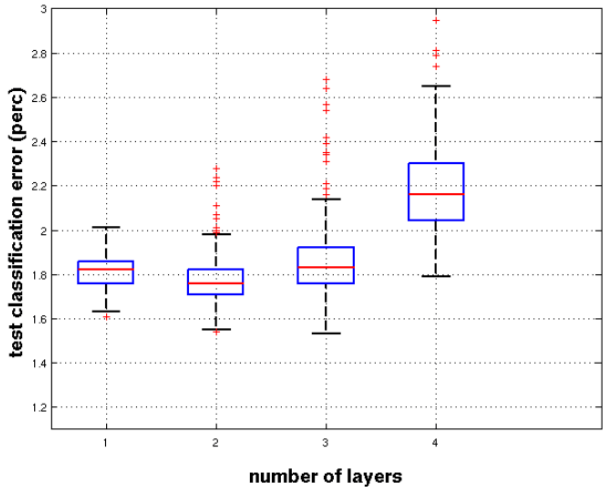
$$\mathscr{L}(\theta) = \frac{1}{m}\sum_{i=1}^{m}(y_i - f(x_i))^2$$

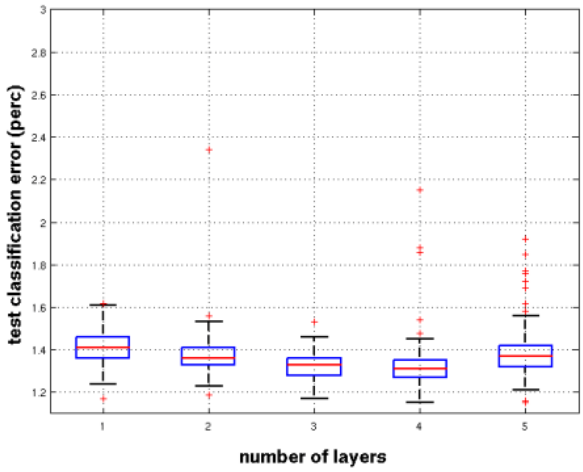
This unsupervised objective ensures that that the learning is not greedy w.r.t. the supervised objective (and also satisfies the unsupervised objective)

Some other experiments have also shown that pre-training is more robust to random initializations

1The difficulty of training deep architectures and effect of unsupervised pre-tra al,2009

One accepted hypothesis is that pretraining leads to better weight initializations (so that the layers capture the internal characteristics of the data)





So what has happened since 2006-2009?

Deep Learning has evolved

Better optimization algorithms

Better regularization methods

Better Activation functions

Better weight initialization strategies

Module 7.3 : Better activation functions

Mitesh M. Khapra

AI4Bharat, Department of Computer Science and Engineering, IIT Madras

Deep Learning has evolved

Better optimization algorithms

Better regularization methods

Better Activation functions

Better weight initialization strategies

Before we look at activation functions, let's try to answer th
question: "What makes Deep Neural Networks powerful ?"

Consider this deep neural network

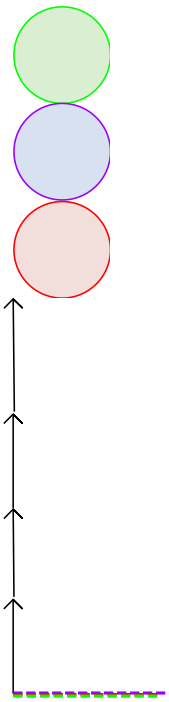
Imagine if we replace the sigmoid in each layer by a simple linear transformation

$y = (w_4 * (w_3 * (w_2 * (w_1 * x))))$

Then we will just learn y as a linear transformation of x

In other words we will be constrained to learning linear decision boundaries

We cannot learn arbitrary decision boundaries



y

h_2

a_3

w_3

a_2

w_2

h_1

a_1

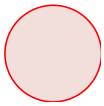
w_1

σ

σ

σ

$h_0 = x$



In particular, a deep linear neural network cannot learn such boundaries

But a deep non linear neural network can indeed learn such boundaries (recall Universal Approximation Theorem)

Now let's look at some non-linear activation functions that are typicall
deep neural networks (Much of this material is taken from Andrej Karpa
notes [1])

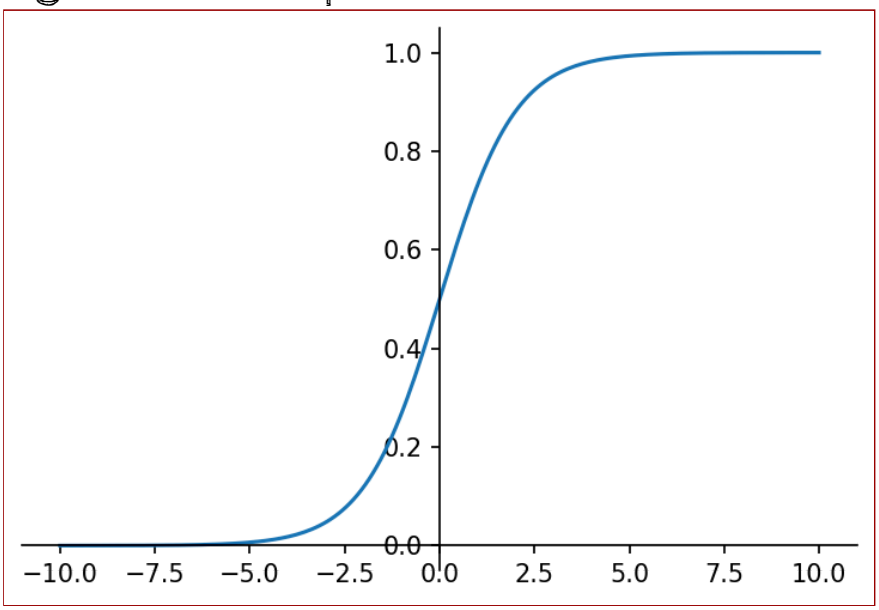
$$\sigma(x) = \frac{1}{1+e^{-x}}$$

As is obvious, the sigmoid function compresses all its inputs to the range [0,1]

Since we are always interested in gradients, let us find the gradient of this function

$$\frac{\partial \sigma(x)}{\partial x} = \sigma(x)(1 - \sigma(x))$$

Let us see what happens if we use sigmoid in a deep network



Sigmoid

x

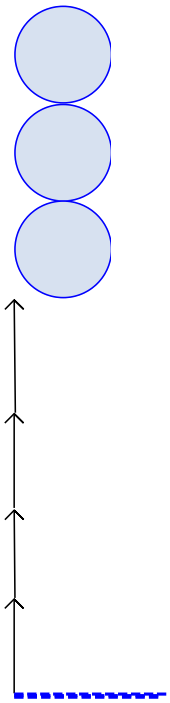
y

While calculating ∇w_2 at some point in the chain rule we will encounter

$$\frac{\partial h3}{\partial a_3} = \frac{\partial \sigma(a_3)}{\partial a_3} = \sigma(a_3)(1 - \sigma(a_3))$$

What is the consequence of this ?

To answer this question let us first understand the concept of saturated neuron ?



a_4

h_2

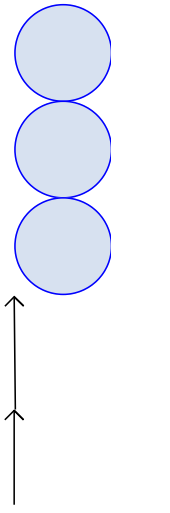
a_1

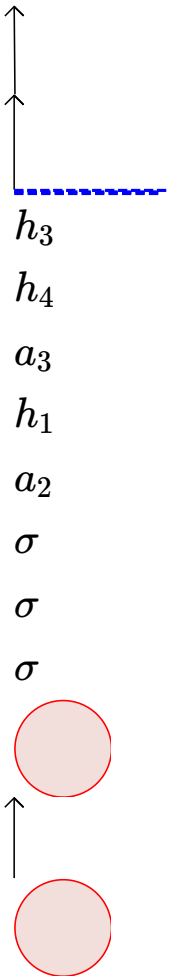
σ

σ

σ

$h_0 = x$

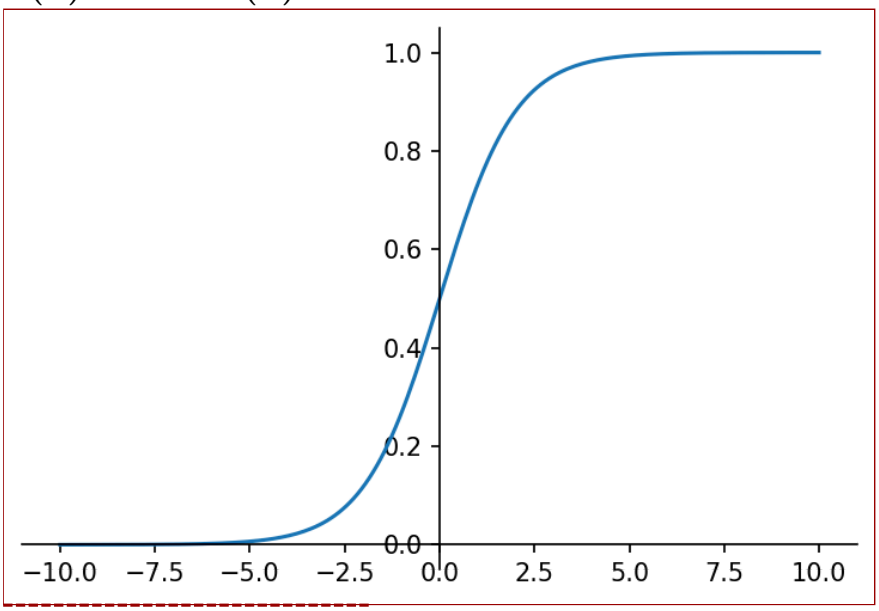




$$a_3 = w_3 h_2$$

$$h_3 = \sigma(a_3)$$

A sigmoid neuron is said to have saturated when $\sigma(x) = 0$ or $\sigma(x) = 1$



What would the gradient be at saturation?

Well it would be 0 (you can see it from the plot or from the formula that we derived)

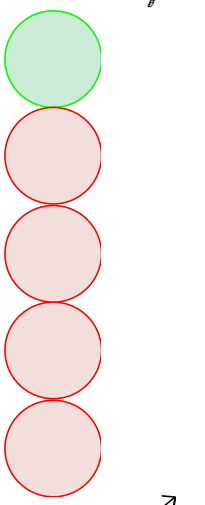
Saturated neurons thus cause the gradient to vanish.

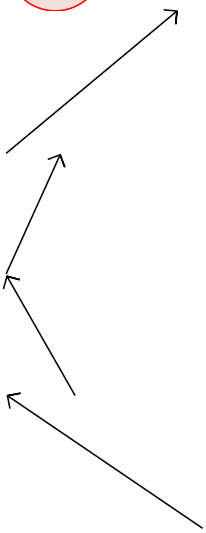
x

y

Saturated neurons thus cause the gradient to vanish.

But why would the neurons saturate ?



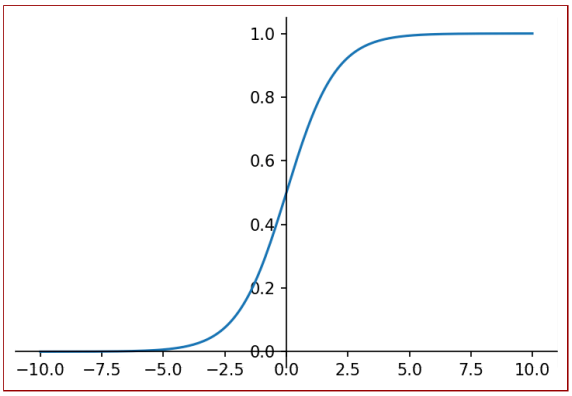


w_1

w_2

w_3

w_4



x

y

Consider what would happen if we use sigmoid neurons and initialize the weights to very high values ?

The neurons will saturate very quickly

The gradients will vanish and the training will stall (more on this later)

$$\sigma(\sum_{i=1}^{n} w_i x_i)$$

Saturated neurons thus cause the gradient to vanish.

Sigmoids are not zero centered

Why is this a problem??

Consider the gradient w.r.t. w_1 and w_2

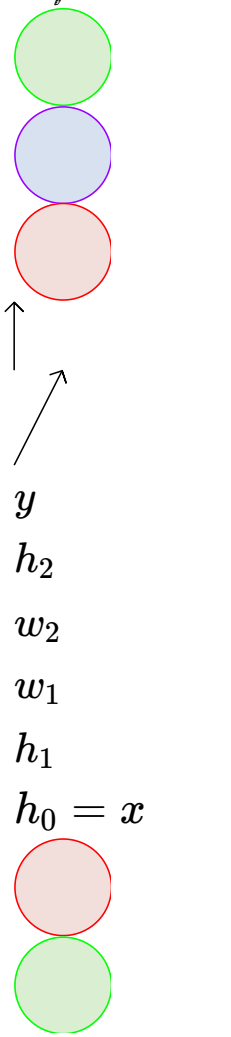
$$\nabla w_1 = \frac{\partial \mathscr{L}(w)}{\partial y} \frac{\partial y}{\partial h_3} \frac{\partial h_3}{\partial a_3} h_{21}$$

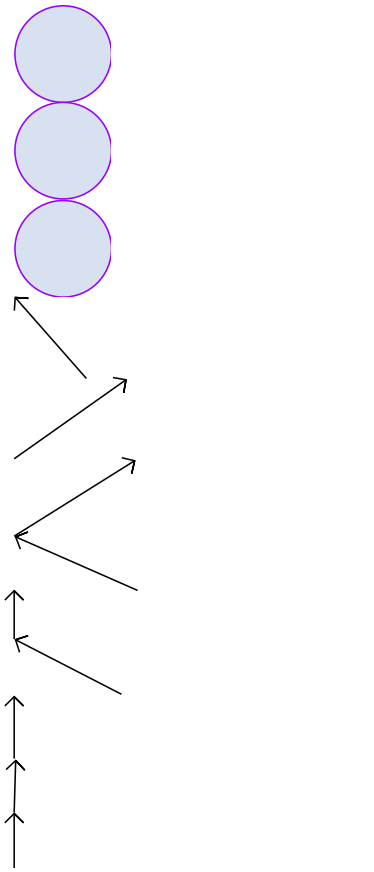
$$\nabla w_2 = \frac{\partial \mathscr{L}(w)}{\partial y} \frac{\partial y}{\partial h_3} \frac{\partial h_3}{\partial a_3} h_{22}$$

Note that h_{21} and h_{22} are between [0, 1] (i.e., they are both positive)

So if the first common term (in red) is positive (negative) then both ∇w_1 and ∇w_2 are positive (negative)

Essentially, either all the gradients at a layer are positive or all the gradients at a layer are negative



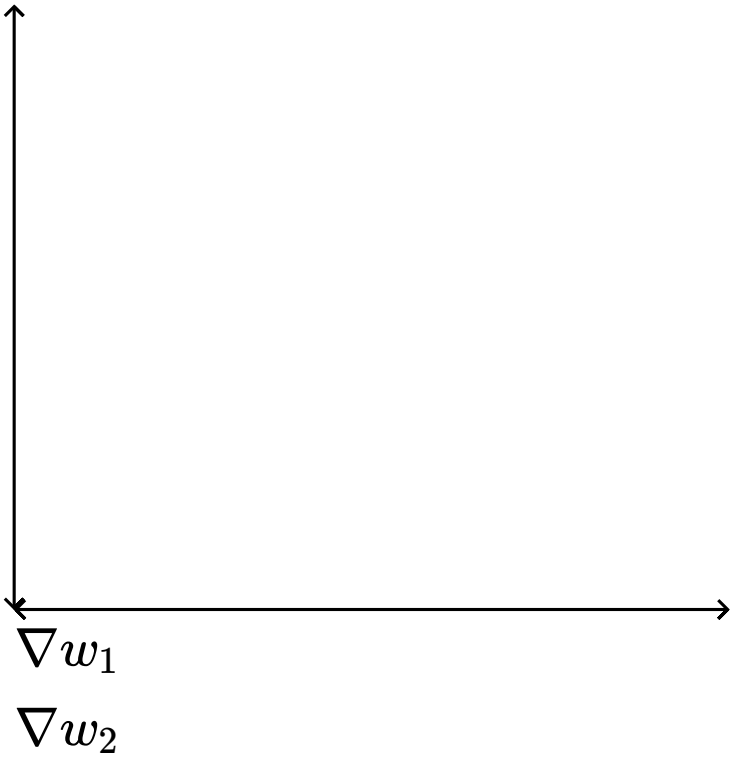


$$a_3 = w_1 h_{21} + w_2 h_{22}$$

Saturated neurons thus cause the gradient to vanish.

Sigmoids are not zero centered

This restricts the possible update directions



Saturated neurons thus cause the gradient to vanish.

Sigmoids are not zero centered

This restricts the possible update directions

∇w_1

∇w_2

(Not possible)

(Not possible)

Quadrant in which all gradients are +ve

(Allowed)

Quadrant in which all gradients are -ve

(Allowed)

Saturated neurons thus cause the gradient to vanish.

Sigmoids are not zero centered

This restricts the possible update directions

∇w_1

∇w_2

(Not possible)

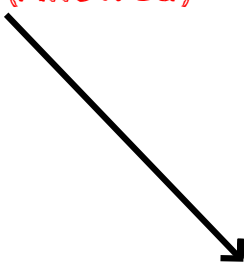
(Not possible)

Quadrant in which all gradients are +ve

(Allowed)

Quadrant in which all gradients are -ve

(Allowed)



Now imagine:

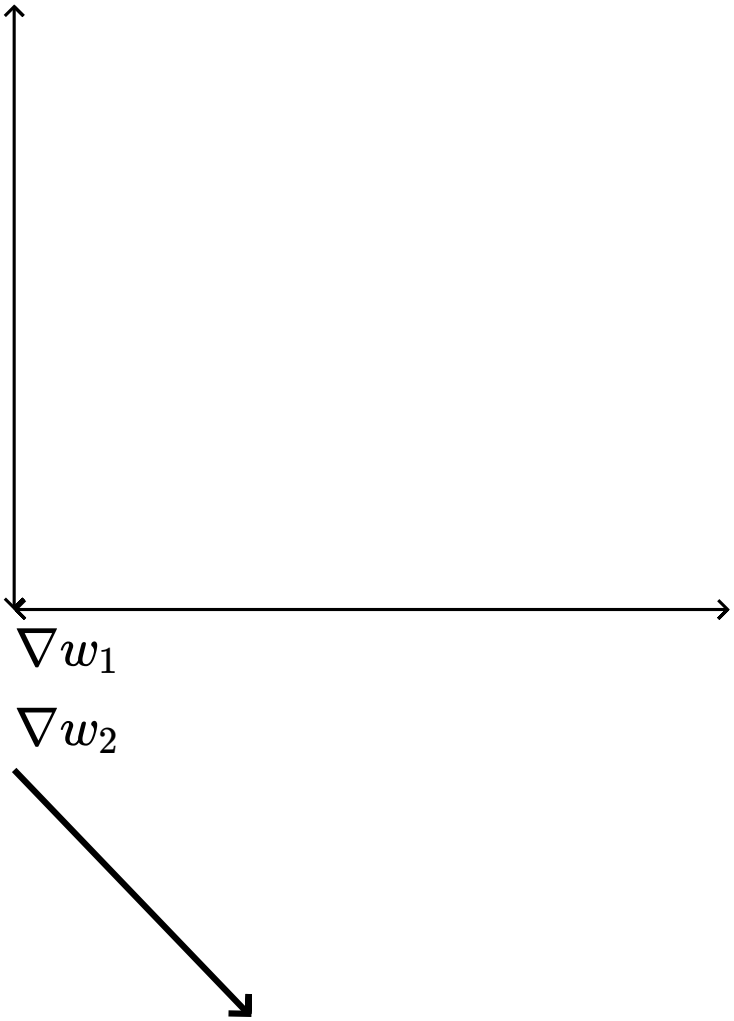
this is the

optimal w

Saturated neurons thus cause the gradient to vanish.

Sigmoids are not zero centered

This restricts the possible update directions



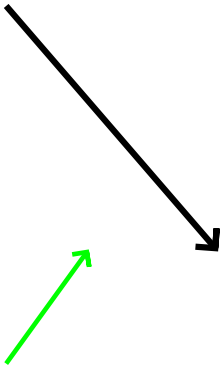
Saturated neurons thus cause the gradient to vanish.

Sigmoids are not zero centered

This restricts the possible update directions

∇w_1

∇w_2



initial position

only way to reach it is

by taking a zigzag path

starting from this

Saturated neurons thus cause the gradient to vanish.

Sigmoids are not zero centered

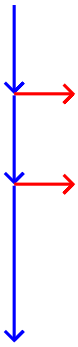
This restricts the possible update directions

initial position

only way to reach it is

by taking a zigzag path

starting from this



And lastly, sigmoids are computationally expensive (because of exp (x))

Compresses all its inputs to the range [-1,1]

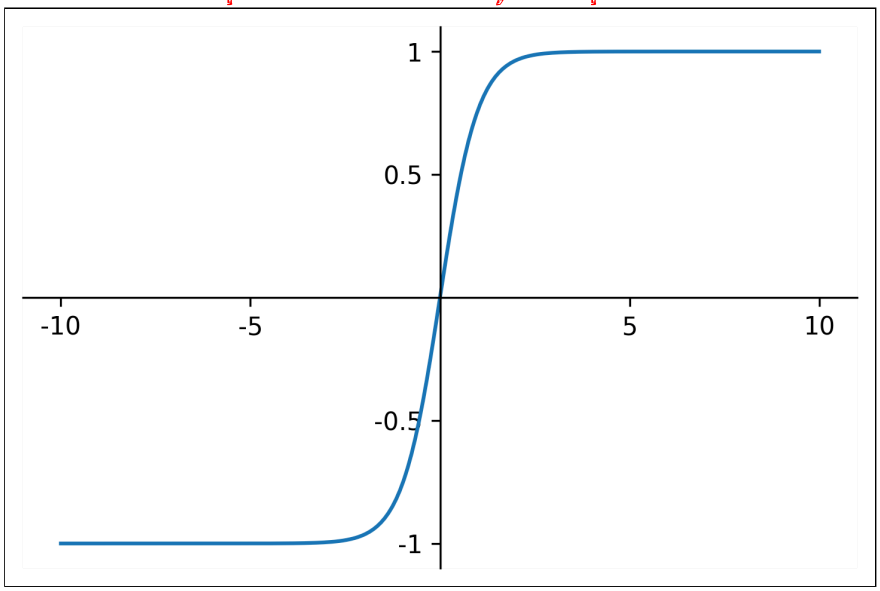
Zero centered

What is the derivative of this function?

$$\frac{\partial tanh(x)}{\partial x} = (1 - tanh^2(x))$$

The gradient still vanishes at saturation

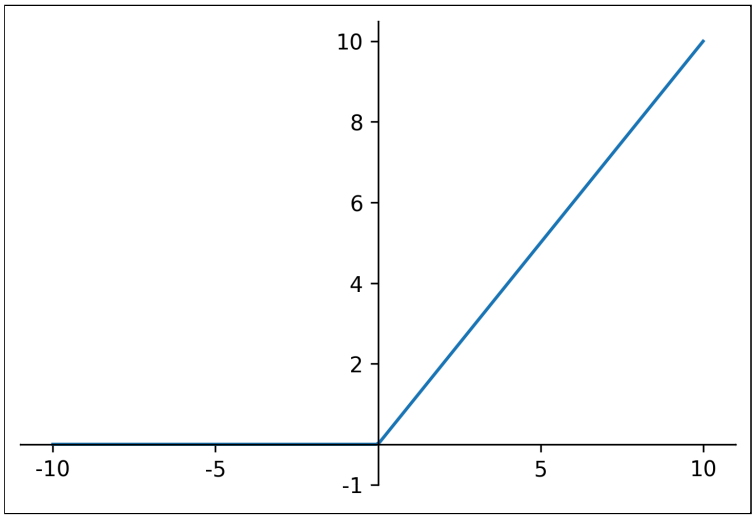
Also computationally expensive



$tanh(x)$

Is this a non-linear function?

$f(x) = max(0, x)$



ReLU (Rectified Linear Unit)

x

$f(x)$

Indeed it is!

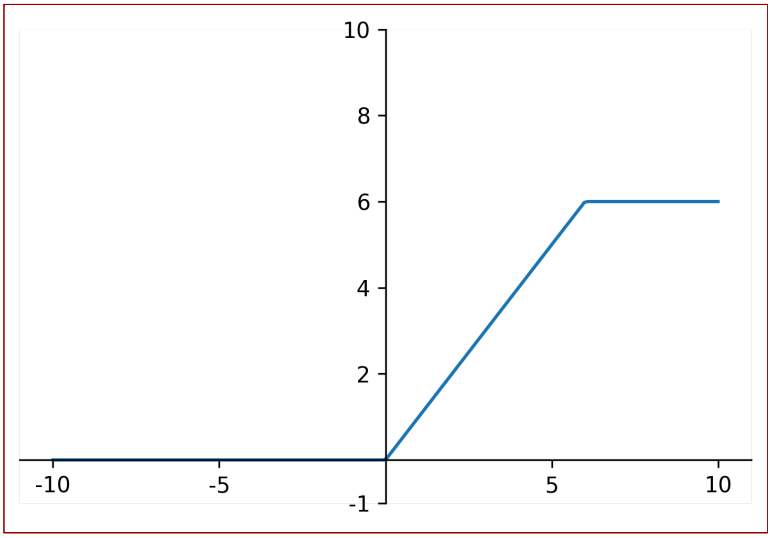
$f(x) = max(0, x) - max(0, x - 6)$

Is this a non-linear function?

Indeed it is!

In fact we can combine two ReLU units to recover a piecewise linear approximation of the scaled sigmoid function

It is also called ReLU6 (6 denotes the maximum value of the ReLU)

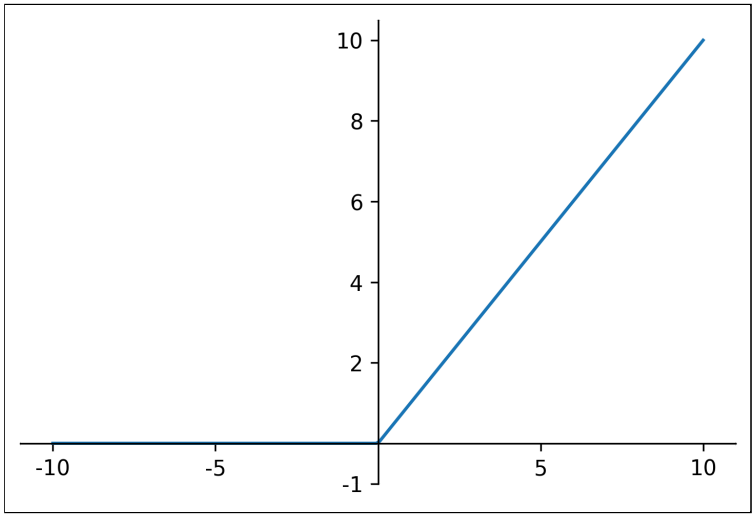


Advantages of ReLU

Does not saturate in the positive region

$f(x) = max(0, x)$

[1]ImageNet Classification with Deep Convolutional Neural Networks- Alex Krizhevsky Ilya Sutskever, G



ReLU (Rectified Linear Unit)

Computationally efficient

In practice converges much faster than $sigmoid/tanh$[1]

In practice there is a caveat

Let's see what is the derivative of ReLU(x)

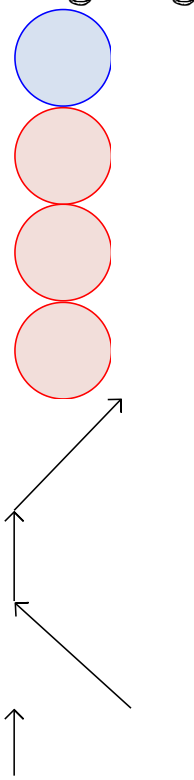
$$\frac{\partial ReLU(x)}{\partial x}$$

$$= 1 \quad if \quad x > 0$$

$$= 0 \quad if \quad x < 0$$

Now consider the given network

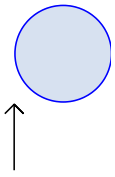
What would happen if at some point a large gradient causes the bias b to be updated to a large negative value?



y

w_3

w_1



a_2

h_1

a_1

w_2

x_1

x_2

b

1

$w_1 x_1 + w_2 x_2 + b < 0 \quad [if \quad b \quad << 0]$

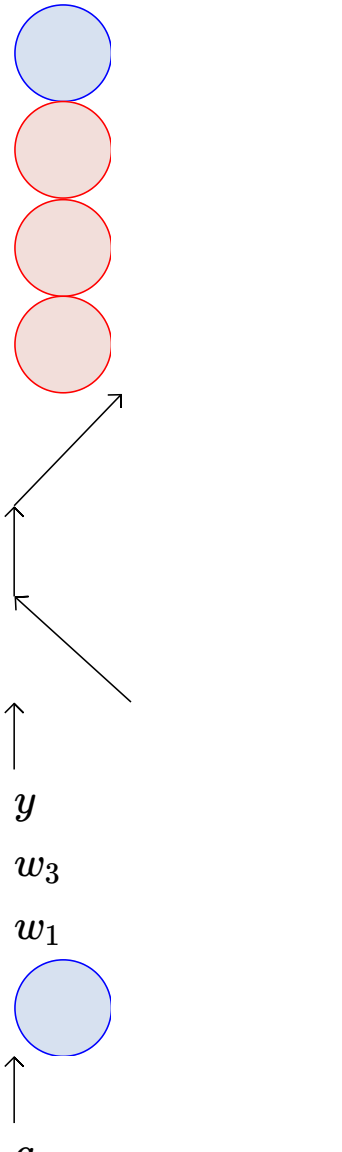
The neuron would output 0 [dead neuron]

Not only would the output be 0 but during backpropagation even the gradient $\frac{\partial h_1}{\partial a_1}$ would be zero

The weights w_1, w_2 and b will not get updated [$\because$ there will be a zero term in the chain rule]

$$\nabla w_1 = \frac{\partial \mathscr{L}(\theta)}{\partial y} \frac{\partial y}{\partial a_2} \frac{\partial a_2}{\partial h_1} \frac{\partial h_1}{\partial a_1} \frac{\partial a_1}{\partial w_1}$$

The neuron will now stay dead forever!!

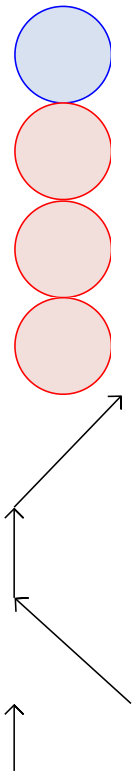


x_2

b

1

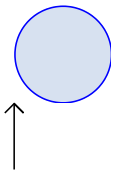
In practice a large fraction of ReLU units can die if the learning rate is set too high
It is advised to initialize the bias to a positive value (0.01)
Use other variants of ReLU (as we will soon see)



y

w_3

w_1



a_2

h_1

a_1

w_2

x_1

x_2

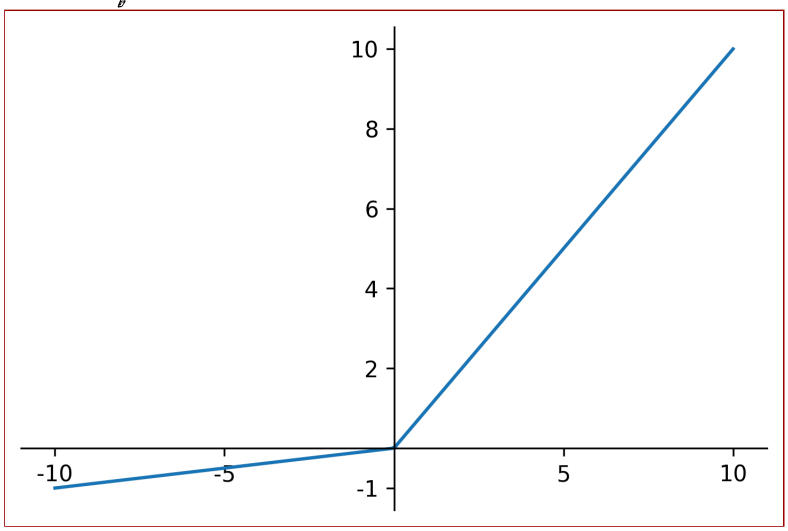
b

1

No saturation

$f(x) = max(0.1x, x)$

Leaky ReLU



Will not die (0.1x ensures that at least a small gradient will flow through)

Computationally efficient

Close to zero centered ouputs

$$f(x) = max(\alpha x, x)$$

α *is a parameter of the model*

α *will get updated during backpropagation*

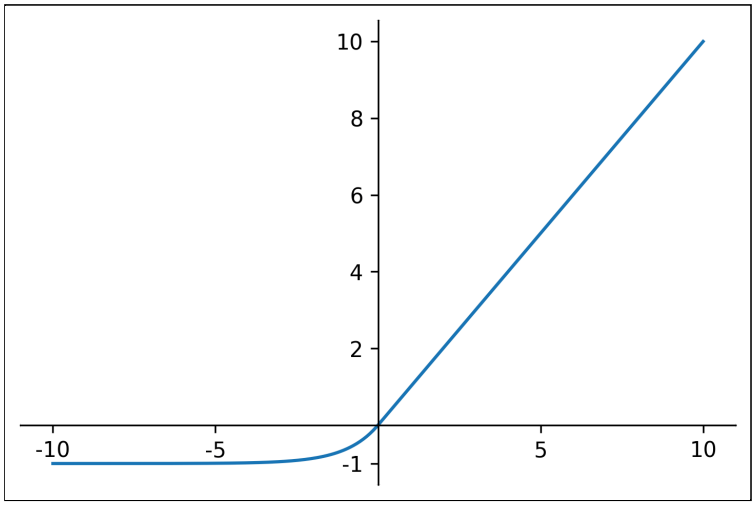
Parametric ReLU

All benefits of ReLU

Exponential Linear Unit (ELU)

$f(x) = x \quad if \quad x > 0$

$= ae^x - 1 \quad if \quad x \leq 0$



$ae^x - 1$ ensures that at least a small gradient will flow through

Close to zero centered outputs

Expensive (requires computation of exp(x))

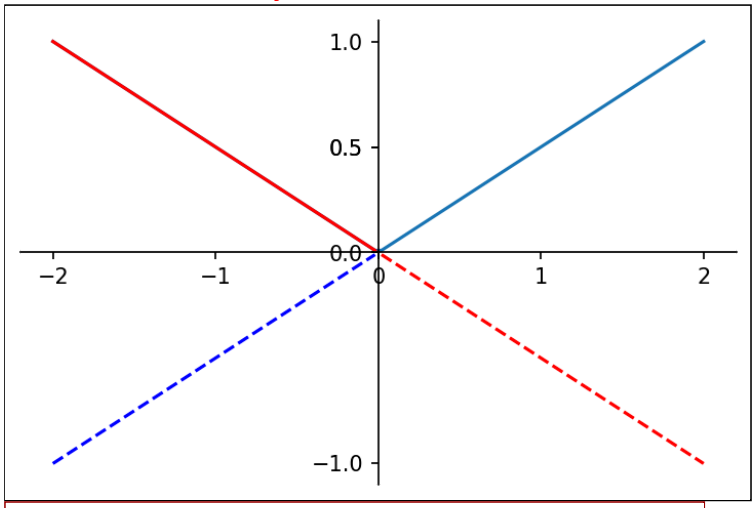
Generalizes ReLU and Leaky ReLU

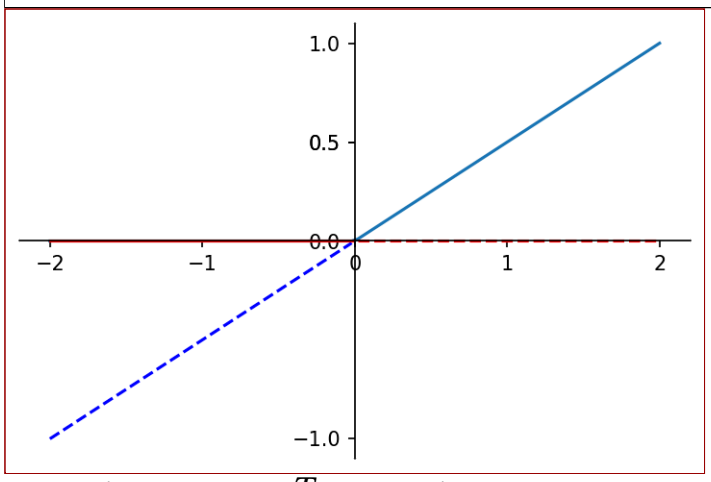
Maxout Neuron: $max(w_1x + b_1, \cdots, w_nx + b_n)$

$max(0.5x, -0.5x)$

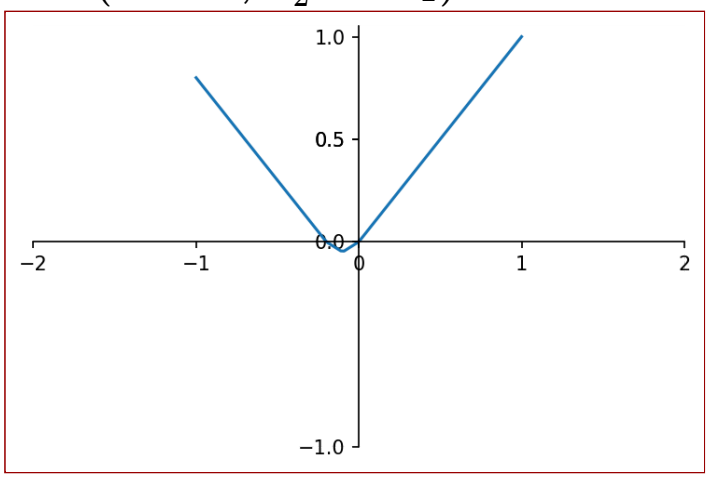
No saturation ! No death!

For k affine transformation in a neuron, k times increase in number of parameters





$max(0x + 0, w_2^Tx + b_2)$

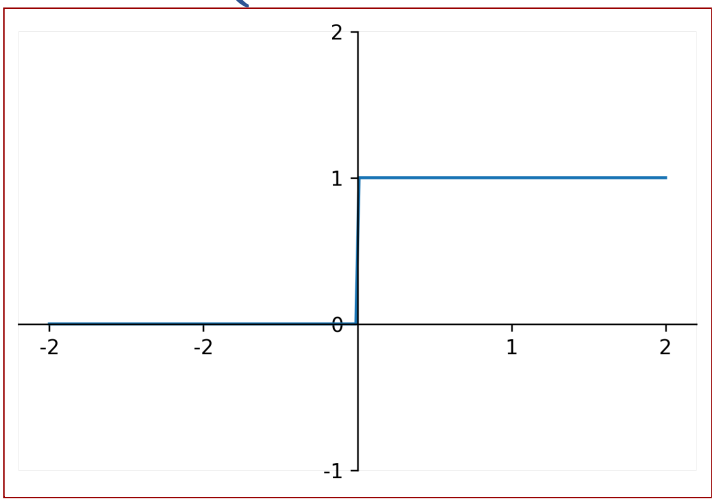


Two MaxOut neurons, with sufficient number of affine transformations, act as a universal approximator!

$max(0.5x, -0.5x, x, -x - 0.2)$

Let's draw some similarities between hard threshold activation function and ReLU activation function

$$f(x) = \begin{cases} 1, & \text{if } x > 0 \\ 0, & x \leq 0 \end{cases}$$



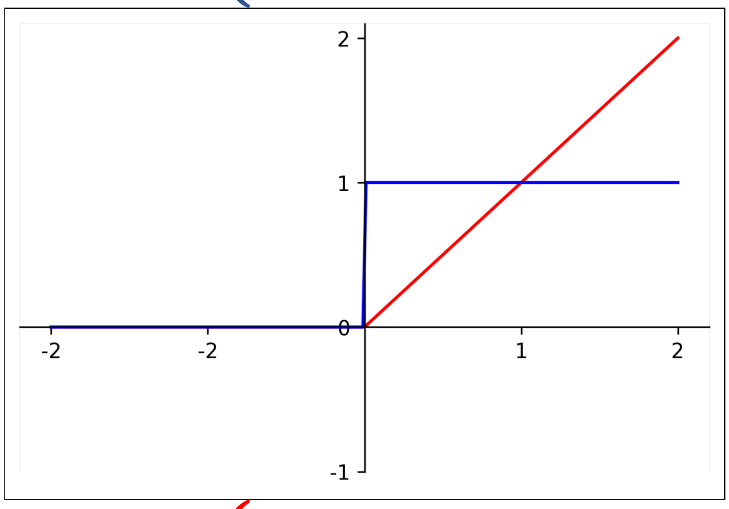
GELU (Gaussian Error Linear Unit)

GELU (Gaussian Error Linear Unit)

Let's draw some similarities between hard threshold activation function and ReLU activation function

Both activation functions output the value based on the sign of the input

$$f(x) = \begin{cases} 1, & \text{if } x > 0 \\ 0, & x \leq 0 \end{cases}$$



$$f(x) = \begin{cases} 1 \cdot x, & \text{if } x > 0 \\ 0, & x \leq 0 \end{cases}$$

Let's rewrite the ReLU activation a bit as follows

$$f(x) = \begin{cases} 1 \cdot x, & \text{if } x > 0 \\ 0 \cdot x, & x \leq 0 \end{cases}$$

The ReLU function multiplies the input by 1 or 0.

The Dropout also does the same.

But with one difference

It does it stochastically as follows

(with a slight abuse of notations)

Therefore, it make sense to combine these properties into the activation function.

$$\mu(x) = \begin{cases} 1 \cdot x, & p \\ 0 \cdot x, & 1-p \end{cases}$$

$$f(x) = m \cdot x$$

$$m \sim Bernoulli(\Phi(x))$$

where,

Now, the multiplication factor m is random and also function of input $\Phi(x)$

The range of $\Phi(x)$ has to be within $0 \leq \Phi(x) \leq 1$

$\Phi(x)$ can be logistic, however, the more natural choice is cumulative distribution of standard normal distribution

The expected value of $f(x) = m.x$ is

$$E(f(x)) = E(m \cdot x)$$

$$= \Phi(x) \cdot 1 \cdot x + (1 - \Phi(x)) \cdot 0 \cdot x$$

$$= \Phi(x)x$$

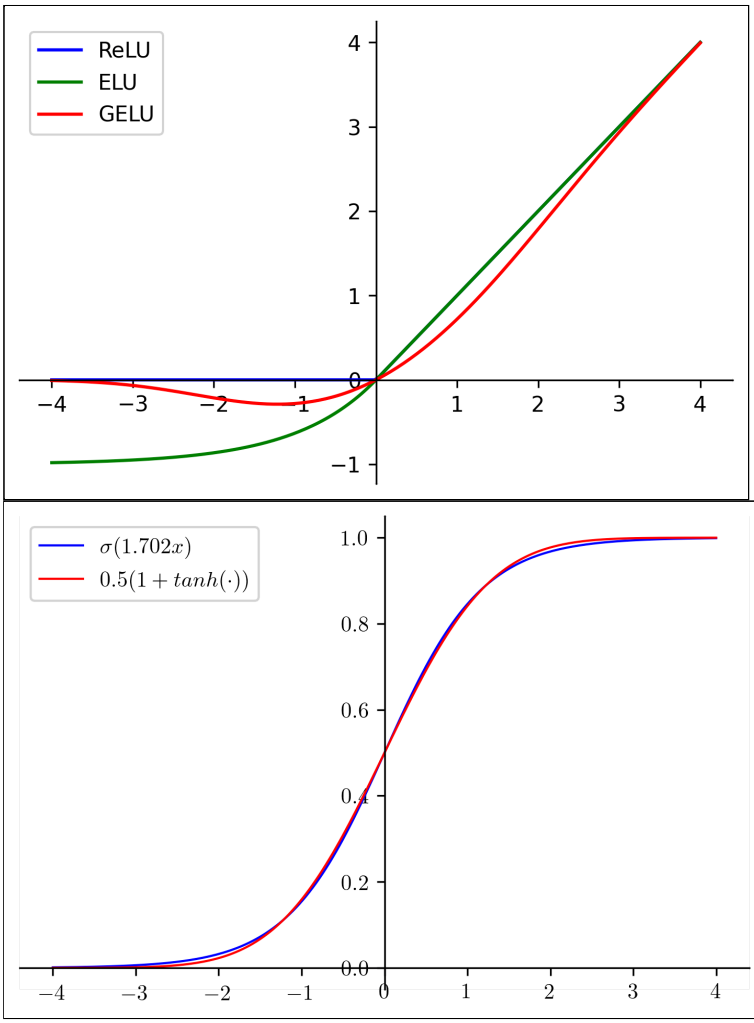
$$= P(X \leq x)x$$

GELU(x)

Often, CDF of Gaussian ($\mu = 0, \sigma = 1$) is computed with the error function.The approximation of error function is given by

$$\approx 0.5x(1 + tanh[\frac{\sqrt{2}}{\sqrt{\pi}}(x + 0.044715x^3)])$$

$$\approx x\sigma(1.702x)$$



SELU (Scaled Exponential Linear Unit)

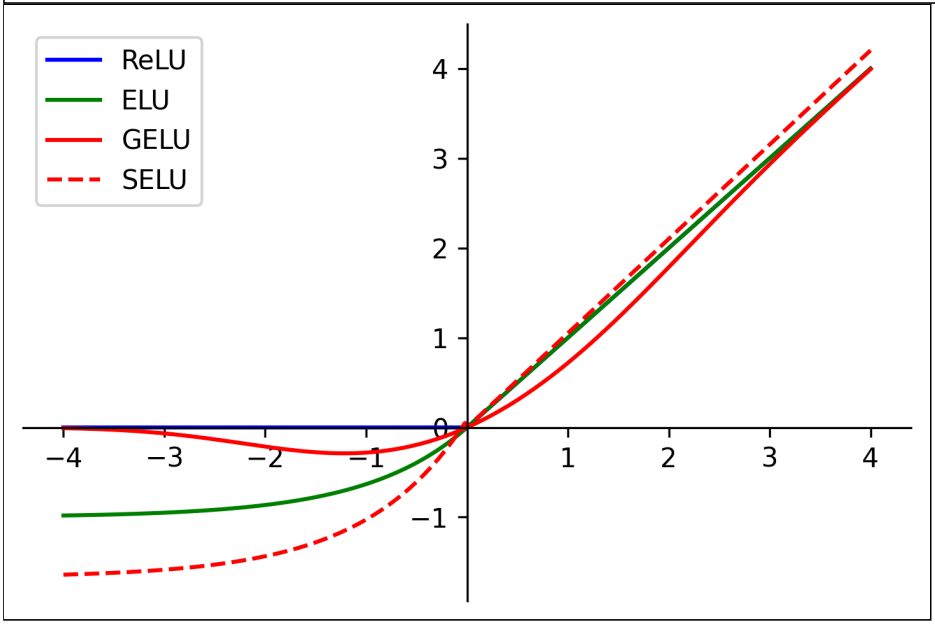
It is helpful to center the output activations from each layer to zero mean and unit variance

We can do this explicitly using normalization techniques (we will see soon)

What if we do this with the activation function itself (called self normalizing)

SELU does it!

$$f(x) = \lambda \begin{cases} x & if \quad x > 0 \\ \alpha e^x - \alpha & if \quad x \leq 0 \end{cases}$$



$\lambda \approx 1.0507009$

$\lambda \approx 1.673263$

Automatic Search for Activation functions

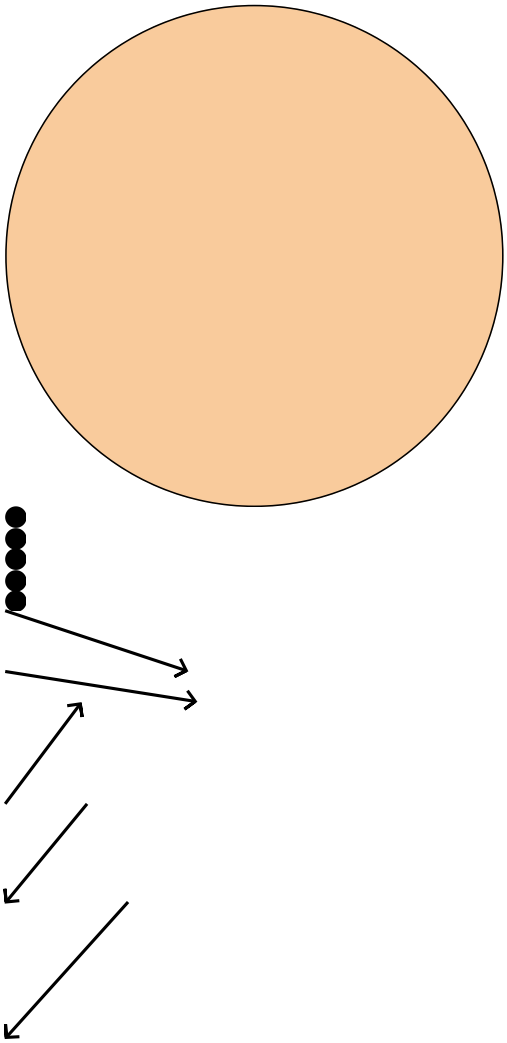
$$\frac{1}{1+e^{(-x)}}$$

$$max(x)$$

$$tanh(x)$$

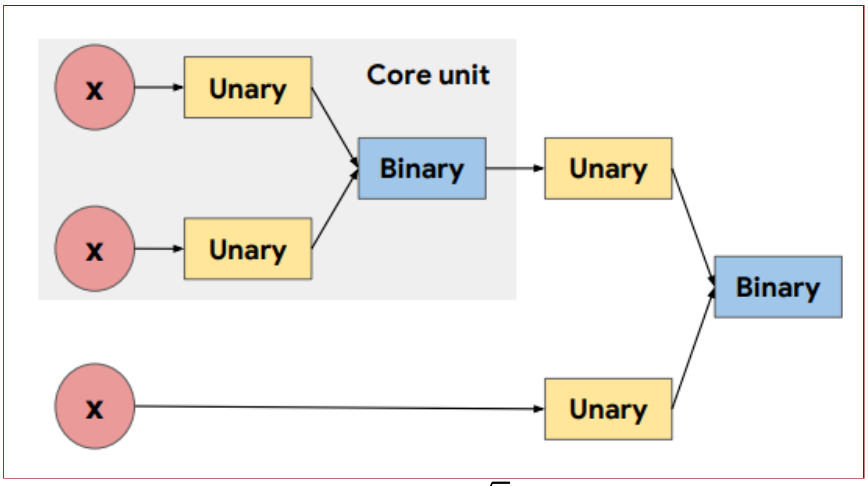
$$x\sigma(1.702x)$$

$$max(\alpha x, x)$$



Assume there exists a search space for activation function (scalar functions)

All such scalar functions are composed of two operations



Unary: $(|x|, -x, exp(-x), \sqrt{(x)})$

Binary: $(max(x_1, x_2), x_1 \cdot \sigma(x_2))$

Then, it is possible to search the space to find the best activation function which improves the accuracy of the existing models

Automatic Search for Activation functions

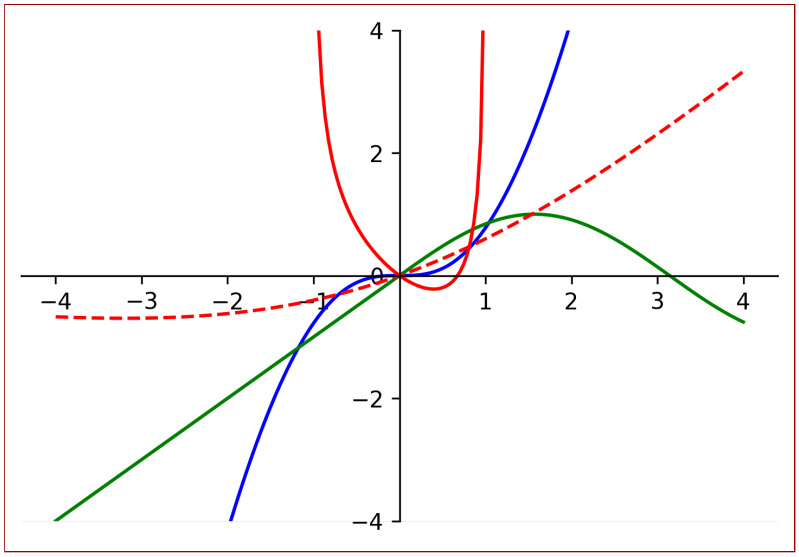
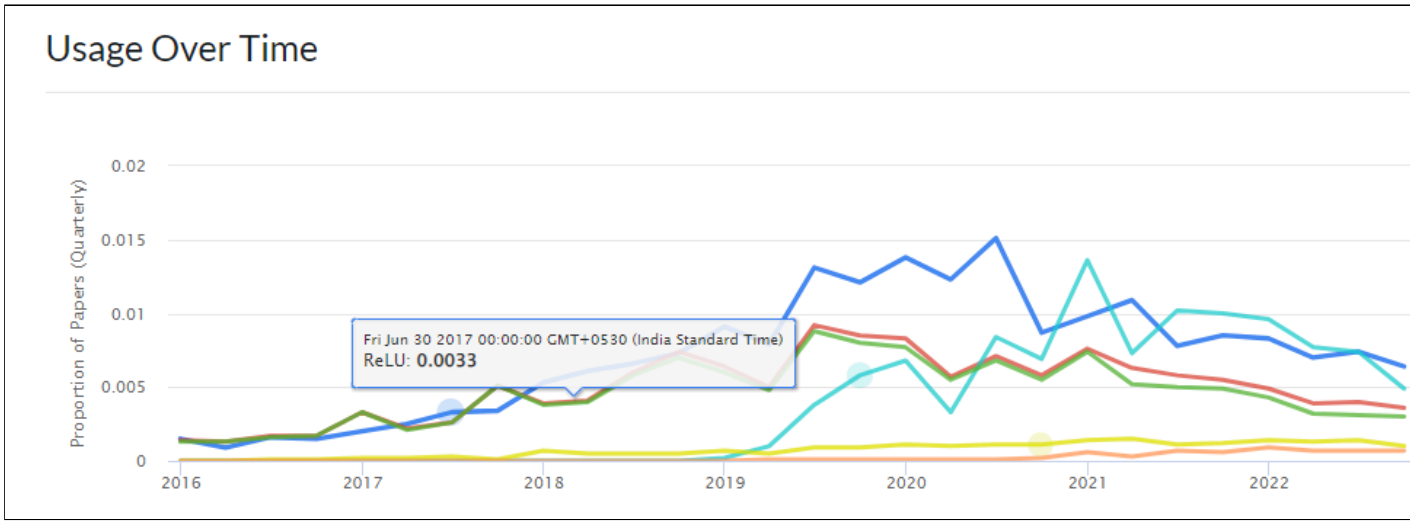


Figure on the right shows top four activation functions found by the search method. According to the search method, the best activation takes the following form

$$f(x) = x\sigma(\beta x)$$

It is named as SWISH. Here, β can be a constant like in GELU $\beta = 1.702$ or a learnable parameter

SILU (Sigmoid-weighted Linear Unit)
We call $x\sigma(1.702x)$ as GELU
We call $x\sigma(\beta x)$ as SWISH
What should we call $x\sigma(x)$?
Interestingly, it was proposed in the Reinforcement learning paradigm


Usage Over Time
Proportion of Papers (Quarterly)
0.02
0.015
0.01
0.005
0
2016
2017
2018
2019
2020
2021
2022
Fri Jun 30 2017 00:00:00 GMT+0530 (India Standard Time)
ReLU: 0.0033

Things to Remember

Sigmoids are bad

ReLU is more or less the standard unit for Convolutional Neural Netw

Can explore Leaky ReLU/Maxout/ELU

tanh sigmoids are still used in LSTMs/RNNs (we will see more on this

GELU is most commonly used in Transformer based architectures like BERT, G

Module 7.4 : Better initialization strategies

Mitesh M. Khapra

AI4Bharat, Department of Computer Science and Engineering, IIT Madras

Deep Learning has evolved

Better optimization algorithms

Better regularization methods

Better activation functions

Better weight initialization strategies

For learning to happen, the Gradients must flow from the Output layer layer.

One of the factors that affects this flow is the saturated neurons in a ne

$$\nabla w = (f(x) - y)\sigma(x)(1 - \sigma(x) * input$$

How do we prevent neurons from getting saturated?

However, initializing the parameters is a function of number of inputs of outputs fan_{out},type of activation function and type of architecture.

By carefully initializing the parameters of the network.

What happens if we initialize all weights to 0?

$a_{11} = w_{11}x_1 + w_{12}x_2$

$a_{12} = w_{21}x_1 + w_{22}x_2$

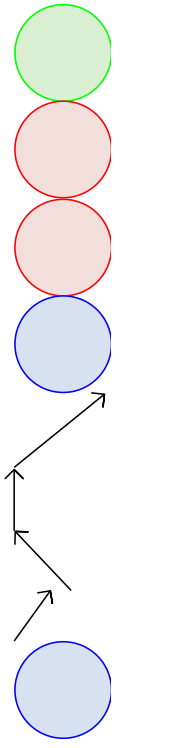
$a_{11} = a_{12} = 0$

$h_{11} = h_{12}$

All neurons in layer 1 will get the same activation

$\therefore$

$\therefore$



y

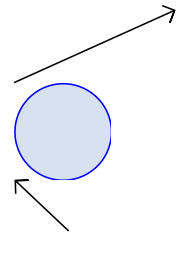
h_{21}

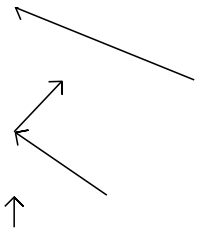
a_{21}

h_{11}

x_1

x_2





a_{11}

h_{13}

a_{13}

h_{12}

a_{12}

σ

σ

σ

σ

Now what will happen during back propagation?

$a_{11} = w_{11}x_1 + w_{12}x_2$

$a_{12} = w_{21}x_1 + w_{22}x_2$

$$\nabla w_{11} = \frac{\partial \mathscr{L}(w)}{\partial y} \cdot \frac{\partial y}{\partial h_{11}} \cdot \frac{\partial h_{11}}{\partial a_{11}} . x_1$$

$$\nabla w_{21} = \frac{\partial \mathscr{L}(w)}{\partial y} \cdot \frac{\partial y}{\partial h_{12}} \cdot \frac{\partial h_{11}}{\partial a_{12}} . x_1$$

$but \quad h_{11} = h_{12}$

$and \quad a_{11} = a_{12}$

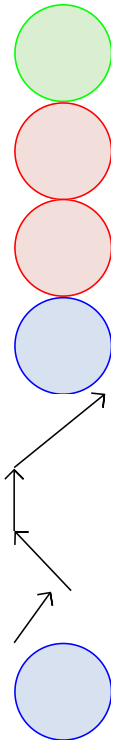
$\nabla w_{11} = \nabla w_{21}$

$a_{11} = a_{12} = 0$

$h_{11} = h_{12}$

$\therefore$

$\therefore$



y

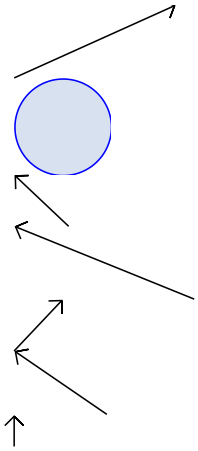
h_{21}

a_{21}

h_{11}

x_1

x_2



a_{11}

h_{13}

a_{13}

h_{12}

a_{12}

σ

σ

σ

σ

$a_{11} = w_{11}x_1 + w_{12}x_2$

$a_{12} = w_{21}x_1 + w_{22}x_2$

Hence both the weights will get the same update and remain equal

Infact this symmetry will never break during training

The same is true for w_{12} and w_{22}

And for all weights in layer 2 (infact, work out the math and convince yourself that all the weights in this layer will remain equal)

This is known as the **symmetry breaking problem**

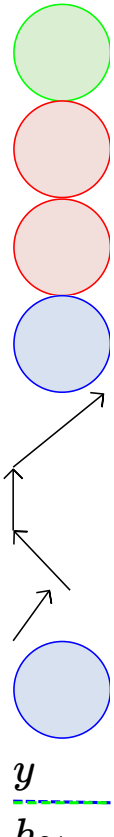
This will happen if all the weights in a network are initialized to the **same value**

$a_{11} = a_{12} = 0$

$h_{11} = h_{12}$

$\therefore$

$\therefore$



y

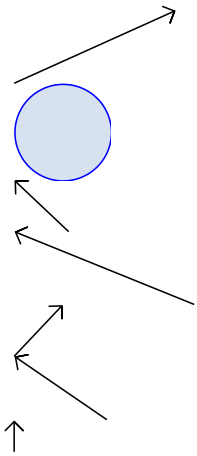
h_{21}

a_{21}

h_{11}

x_1

x_2



a_{11}

h_{13}

a_{13}

h_{12}

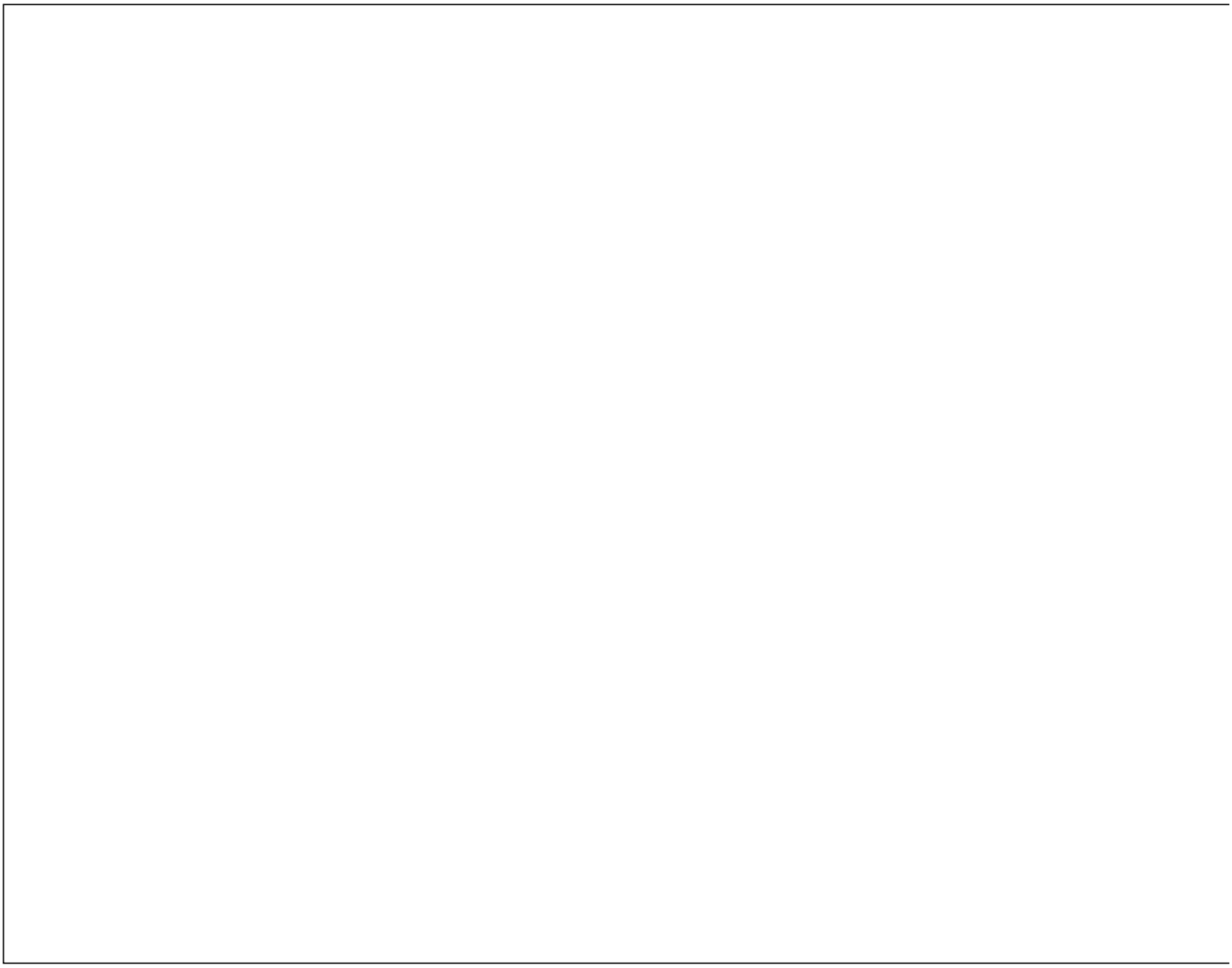
a_{12}

σ

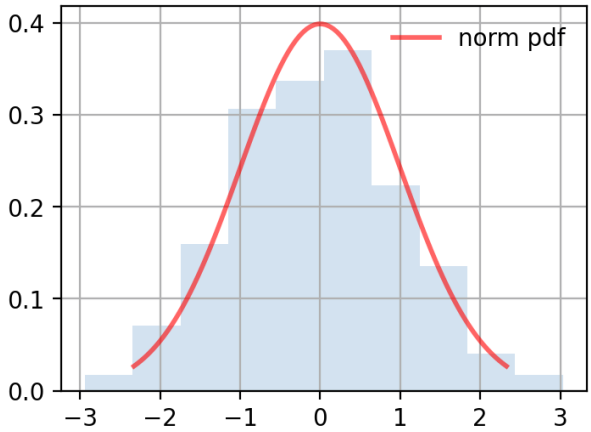
σ

σ

σ



We will now consider a feedforward network with:
input: 1000 points, each $\in R^{500}$
input data is drawn from unit Gaussian
the network has 5 layers
each layer has 500 neurons
we will run forward propagation on this network with different weight initializations



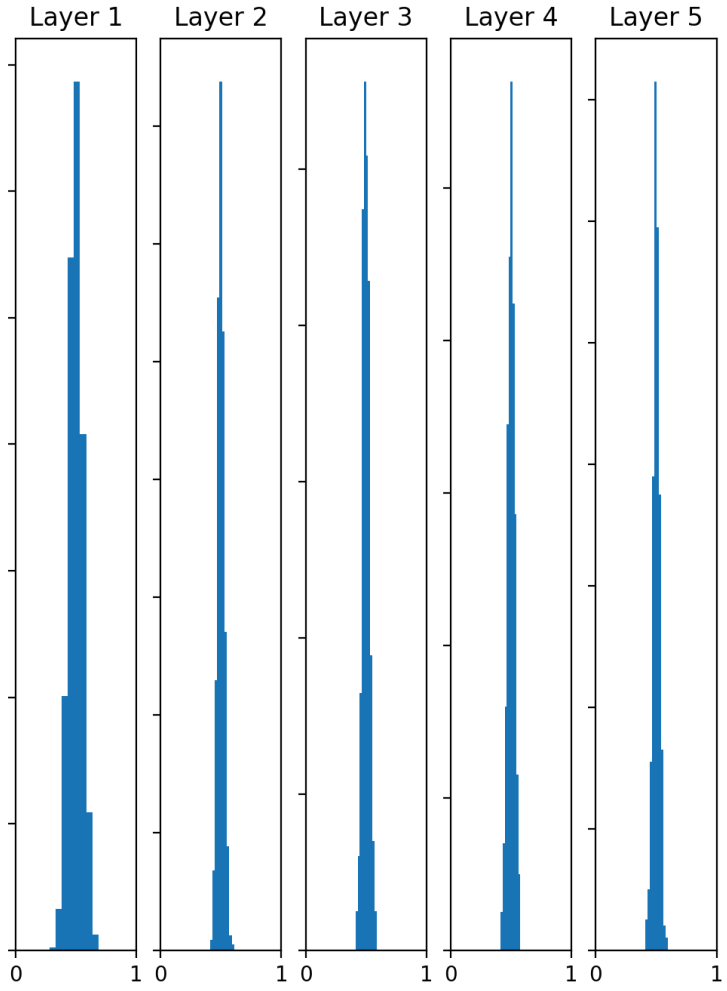

```
num_layers = 5
D = np.random.randn(1000,500)
for i in range(num_layers):
  x = D if i==0 else h_cache[i-1]
  W = 0.01*np.random.randn(500,500)
  a = np.dot(x,W)
  h = sigmoid(a)
```


```
num_layers = 5
D = np.random.randn(1000,500)
for i in range(num_layers):
  x = D if i==0 else h_cache[i-1]
  W = 0.01*np.random.randn(500,500)
  a = np.dot(x,W)
  h = sigmoid(a)
```

Let's try to initialize the weights to small random numbers

We will see what happens to the activation across different layers with sigmoid activation functions

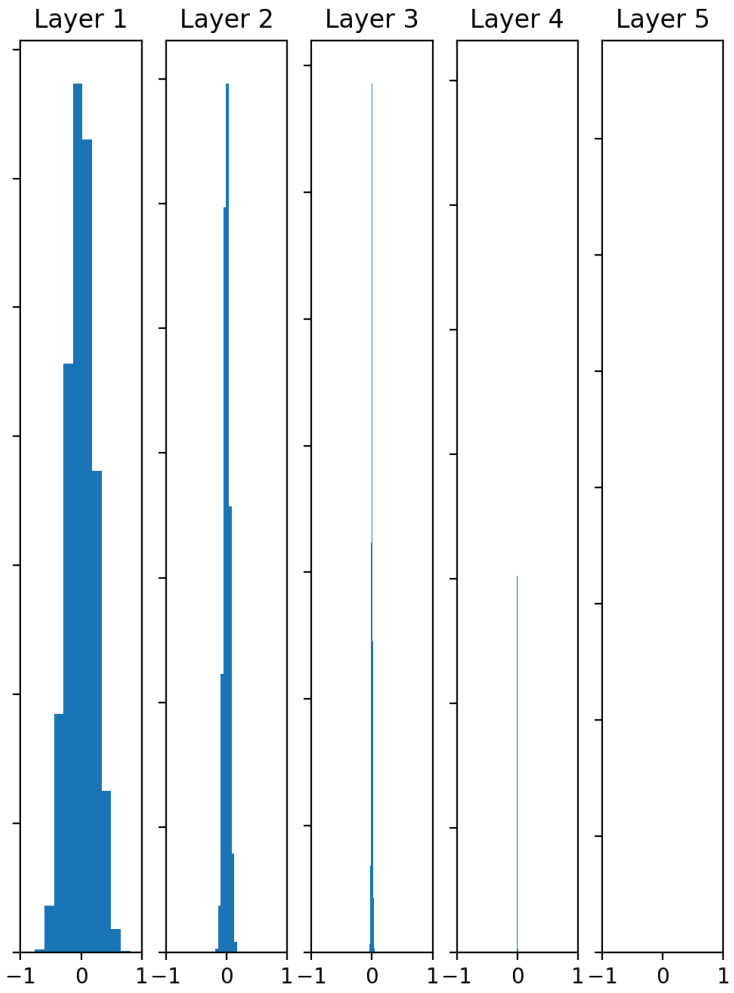
```
W = 0.01*np.random.randn(500,500)
```



Let's try to initialize the weights to small random numbers

We will see what happens to the activation across different layers with tanh activation functions

```
W = 0.01*np.random.randn(500,500)
```

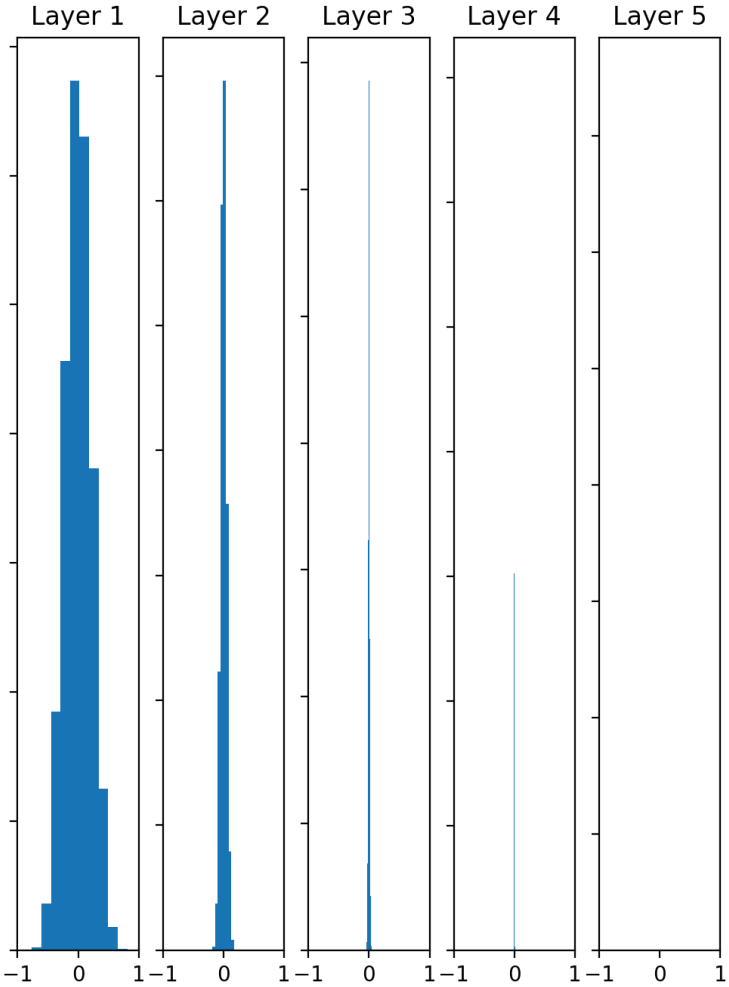


What will happen during back propagation?

Recall that ∇w_1 is proportional to the activation passing through it

If all the activations in a layer are very close to 0, what will happen to the gradient of the weights connecting this layer to the next layer?

They will all be close to 0 (vanishing gradient problem)



Now, let's experiment with the real world data and see what happens

Data: MNIST, $x \in \mathbb{R}^{784}$

Model: FFNN with three hidden layers

$h_1, h_2, h_3 \in \mathbb{R}^{500}$

Loss: Cross entropy

Optimizer: Mini-Batch GD

$\eta = 0.5$

Batch size: 256

Activation: tanh

h_1

h_2

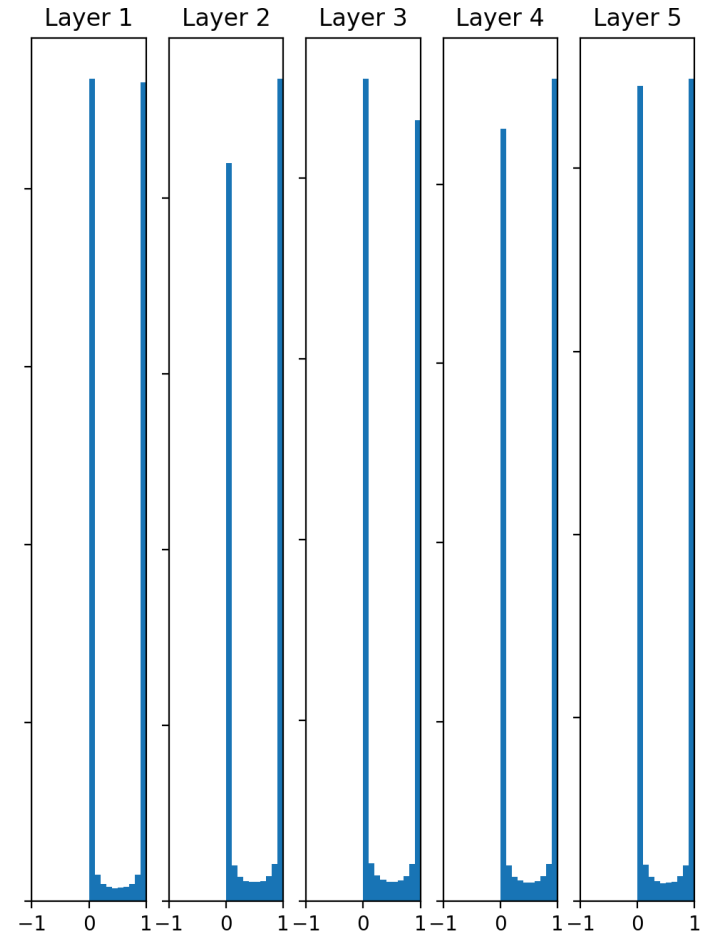
h_3

$\mathscr{L}(\theta)$

```
W = 0.01*np.random.randn(500,500)
```

Observe that for the first 35 iterations, the loss (almost) remains constant

Let us try to initialize the weights to large random numbers



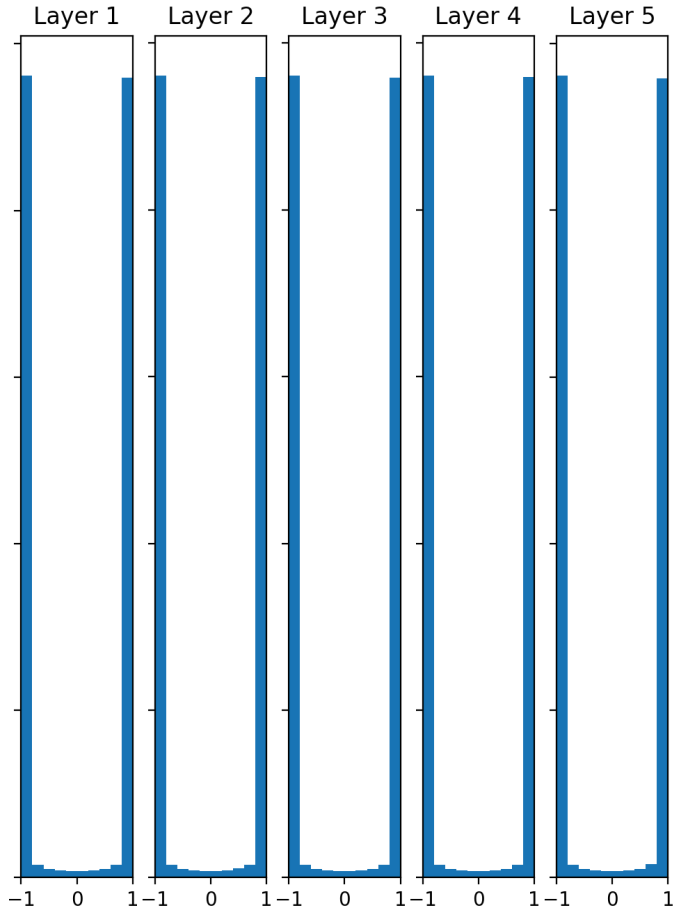

```
W = np.random.randn(500,500)
```

$\sigma()$

Let us try to initialize the weights to large random numbers

```
W = np.random.randn(500,500)
```

tanh()



Most activations have saturated

What happens to the gradients at saturation?

They will all be close to 0 (vanishing gradient problem)

Now, let's experiment with the real world data and see what happens

Data: MNIST, $x \in \mathbb{R}^{784}$

Model: FFNN with three hidden layers

$h_1, h_2, h_3 \in \mathbb{R}^{500}$

Loss: Cross entropy

Optimizer: Mini-Batch GD

$\eta = 0.5$

Batch size: 256

Activation: tanh

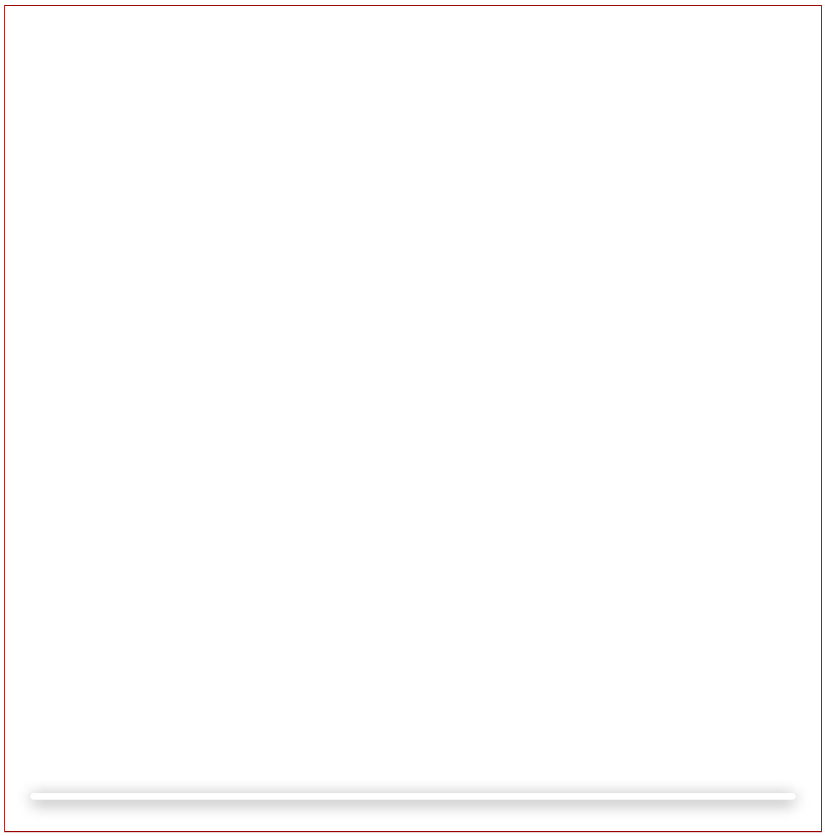
h_1

h_2

h_3

$\mathscr{L}(\theta)$

```
W = np.random.randn(500,500)
```



Let us try to arrive at a more principled way of initializing weights

$$s_{11} = \sum_{i=1}^{n} w_{1i} x_i$$

$$\text{Var}(s_{11}) = \text{Var} \sum_{i=1}^{n} w_{1i} x_i = \sum_{i=1}^{n} \text{Var}(w_{1i} x_i)$$

$$+(E[x_i])^2 Var(w_{1i}) + Var(x_i) Var(w_{1i})]$$

$$= \sum_{i=1}^{n} [(E[w_{1i}])^2 Var(x_i)$$

[Assuming 0 Mean inputs and weights]

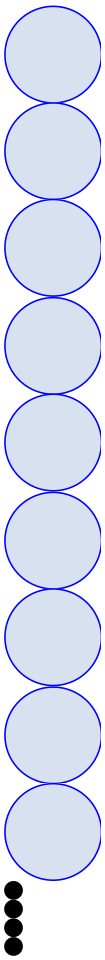
$$= \sum_{i=1}^{n} Var(x_i) Var(w_{1i})$$

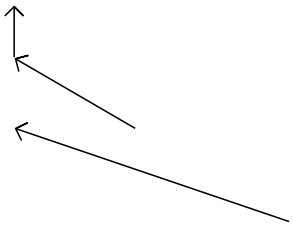
[Assuming $Var(xi) = Var(x) \forall i$]

[Assuming

$$Var(w_{1i}) = Var(w) \forall i]$$

$$= (nVar(w))(Var(x))$$





s_{11}

s_{1n}

x_1

x_2

x_3

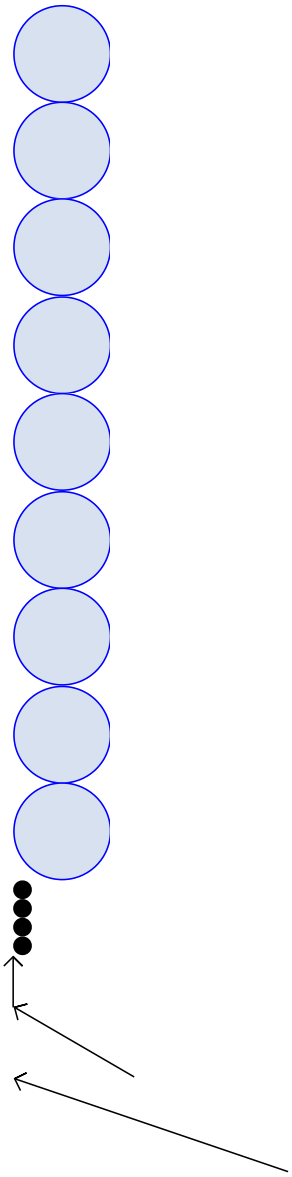
x_n

In general,

$Var(S_{1i}) = (nVar(w))(Var(x))$

What would happen if $nVar(w) \gg 1$

?



s_{11}

s_{1n}

x_1

x_2

x_3

x_n

The variance of S_{1i} will be large

What would happen if $nVar(w) \rightarrow 0$?

The variance of S_{1i} will be small

Let us see what happens if we add one more layer

Using the same procedure as above we will arrive at

$$Var(s_{21}) = \sum_{i=1}^{n} Var(s_{1i})Var(w_{2i})$$

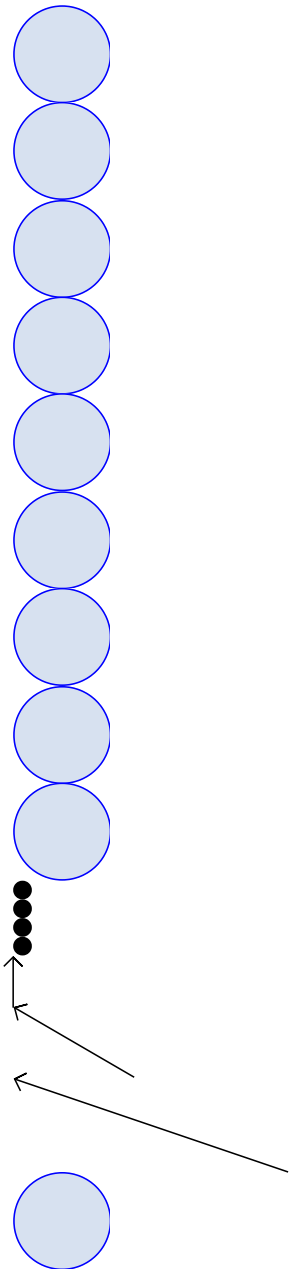
$$= nVar(s_{1i})Var(w_2)$$

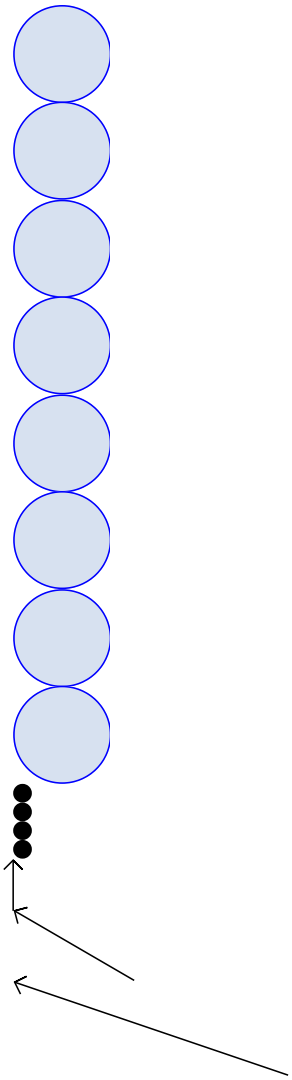
$$\boxed{Var(S_{i1}) = nVar(w_1)Var(x)}$$

$$Var(s_{21}) \propto [nVar(w_2)][nVar(w_1)]Var(x)$$

$$\propto [nVar(w)]^2 Var(x)$$

Assuming weights across all layers have the same variance





s_{21}

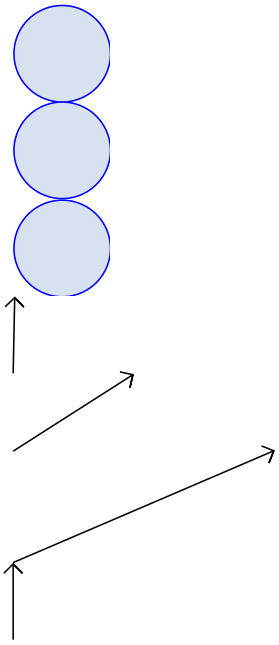
s_{1n}

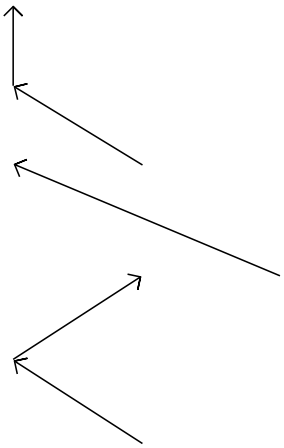
x_1

x_2

x_3

x_n





s_{11}

In general,

$$Var(s_{ki}) = [nVar(w)]^k Var(x)$$

To ensure that variance in the output of any layer does not blow up or shrink we want:

$$nVar(w) = 1$$

If we draw the weights from a unit Gaussianand scale them by then, we have , $\frac{1}{\sqrt{n}}$

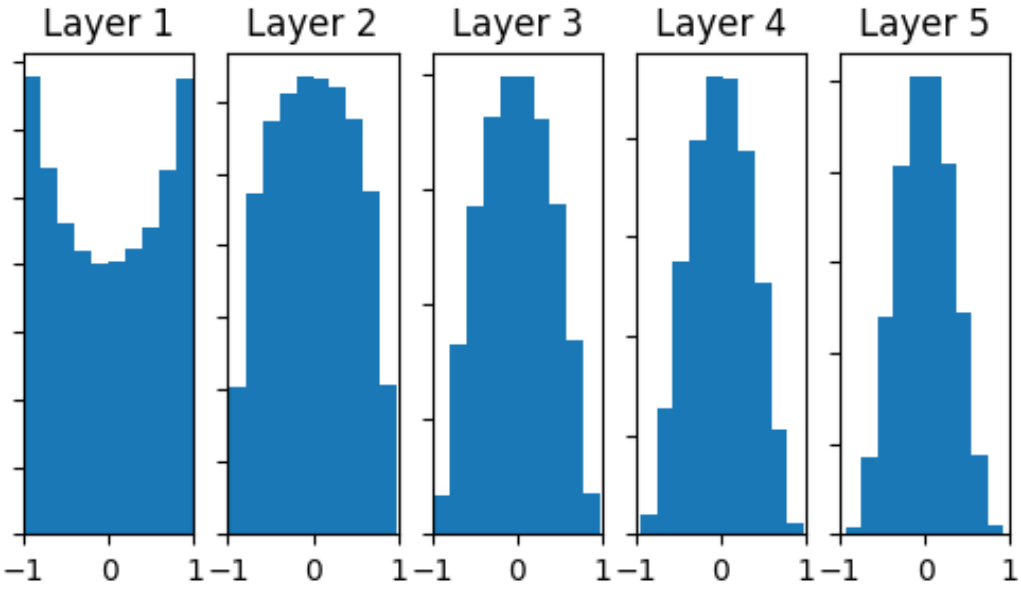
$$nVar(w) = nVar(\frac{z}{\sqrt{n}})$$

$$= n * \frac{1}{n} Var(z) = 1$$

$$\boxed{Var(az) = a^2(Var(z))}$$

← (UnitGaussian)

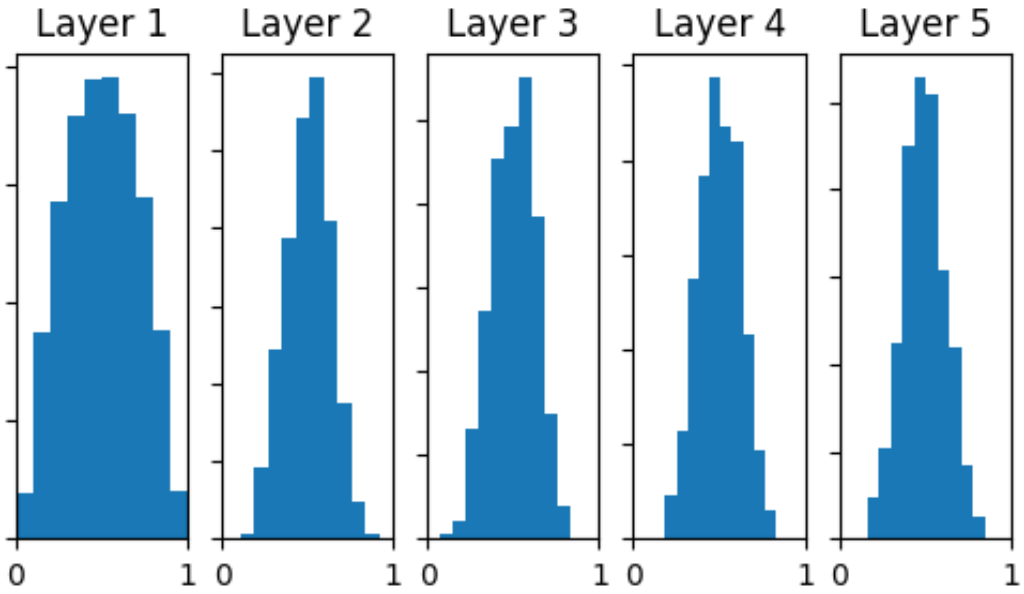
Let's see what happens if we use this initialization



$\tanh(h_i)$

Let's see what happens if we use this initialization

$\sigma(h_i)$



Now, let's experiment with the real world data and see what happens

Data: MNIST, $x \in \mathbb{R}^{784}$

Model: FFNN with three hidden layers

$h_1, h_2, h_3 \in \mathbb{R}^{500}$

Loss: Cross entropy

Optimizer: Mini-Batch GD

$\eta = 0.5$

Batch size: 256

Activation: tanh

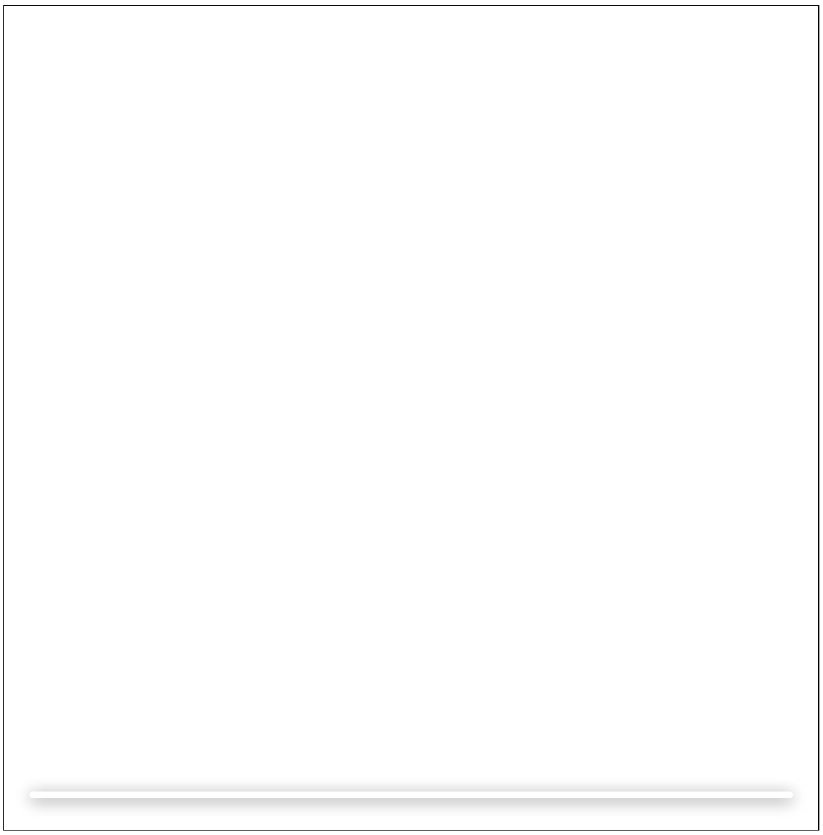
h_1

h_2

h_3

$\mathscr{L}(\theta)$

```
W = np.random.randn(500,500)/ sqrt(fan_in)
```

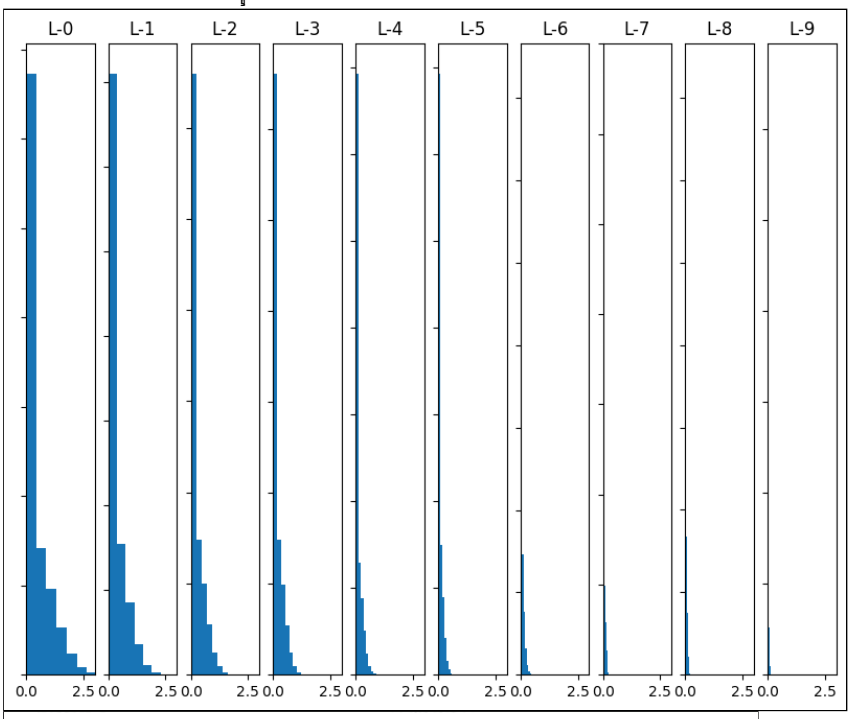


However this does not work for ReLU neurons

Why ?

Intuition: *He et.al.* argue that a factor of 2 is needed when dealing with ReLU Neurons

Intuitively this happens because the range of ReLU neurons is restricted only to the positive half of the space



380 iterations

$h_1 \in \mathbb{R}^{100}$

$h_2, .., h_7 \in \mathbb{R}^{50}$

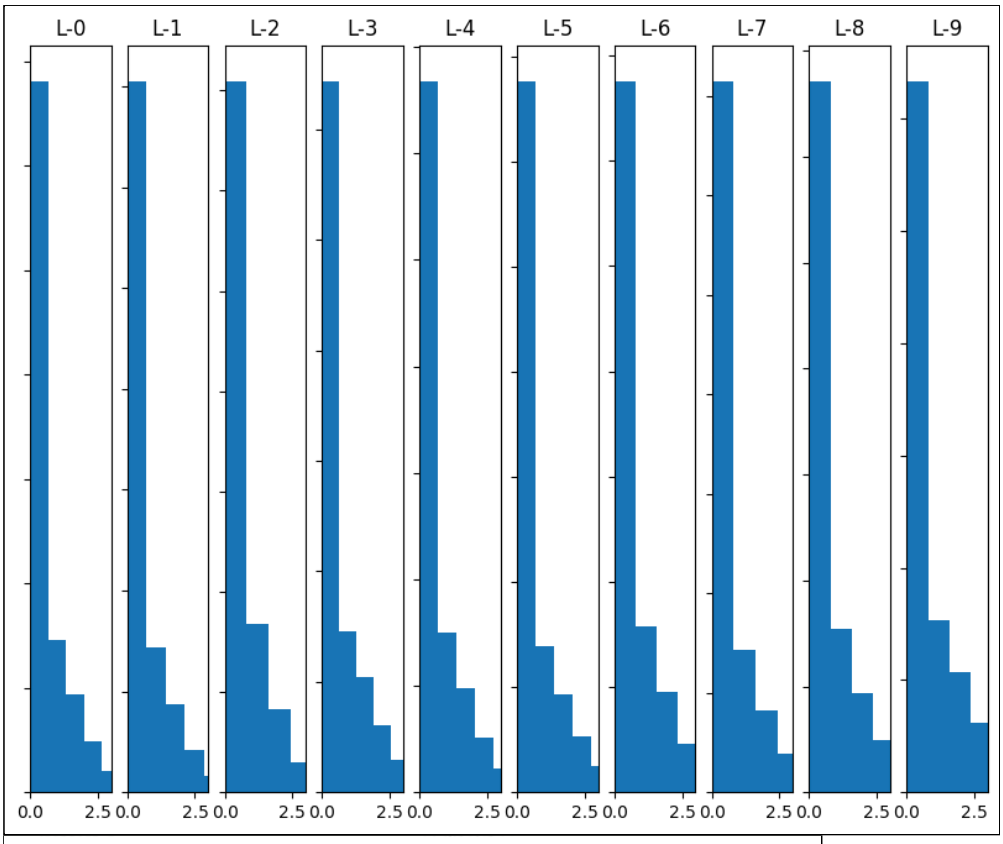
h_1

h_4

h_7

30 iterations/s

Indeed when we account for this factor of 2 we see better performance



Module 7.5 : Batch Normalization

Mitesh M. Khapra

AI4Bharat, Department of Computer Science and Engineering, IIT Madras

We will now see a method called batch normalization whic
less careful about initialization

To understand the intuition behind Batch Normalization let us consider a deep network

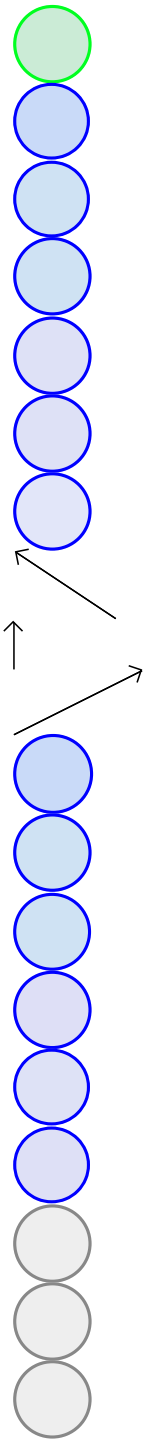
Let us focus on the learning process for the weights between these two layers

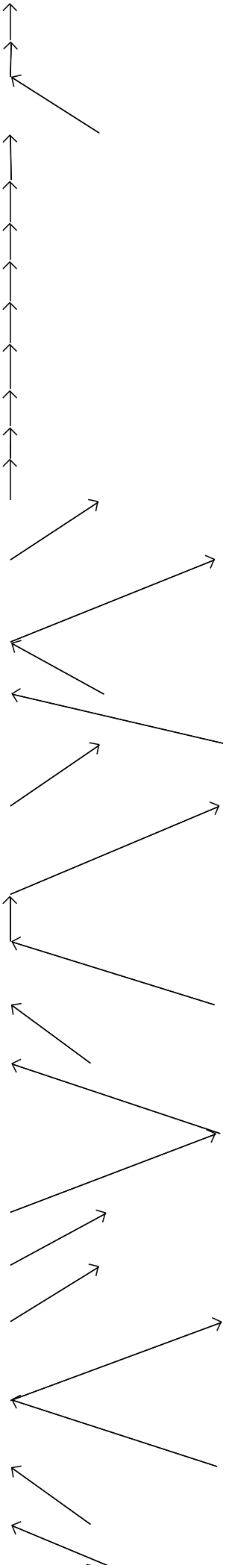
Typically we use mini-batch algorithms

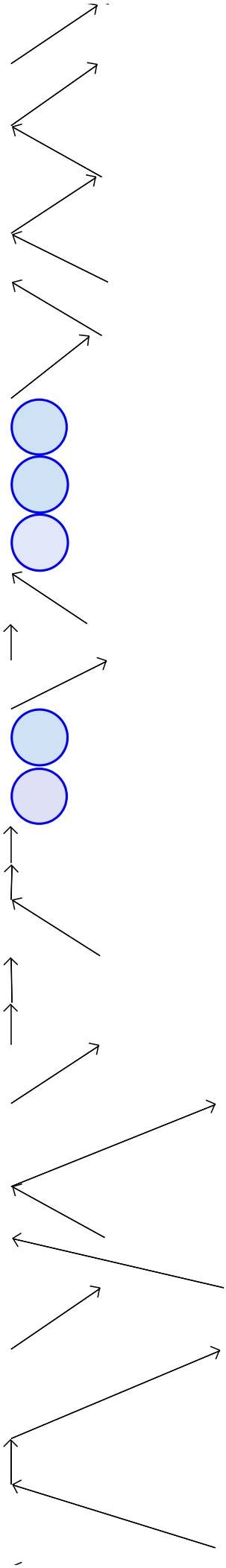
What would happen if there is a constant change in the distribution of h_3

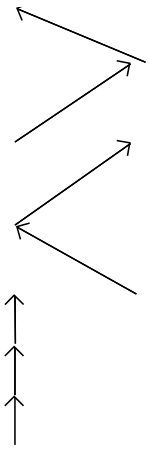
In other words what would happen if across minibatches the distribution of h_3 keeps changing

Would the learning process be easy or hard?









x_1

x_2

x_3

h_4

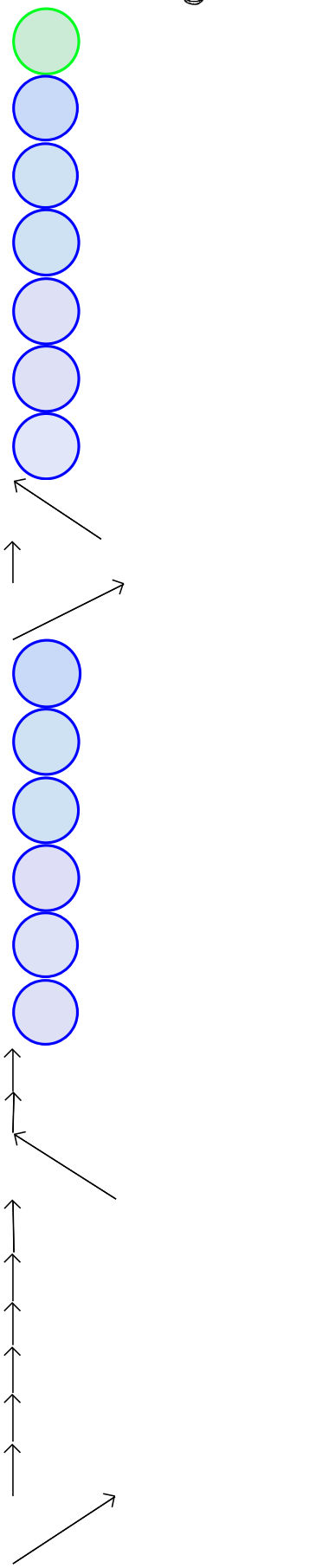
h_3

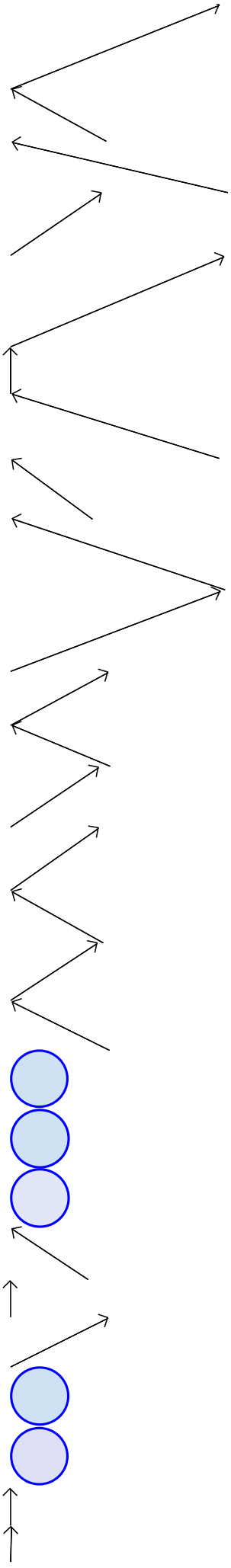
h_2

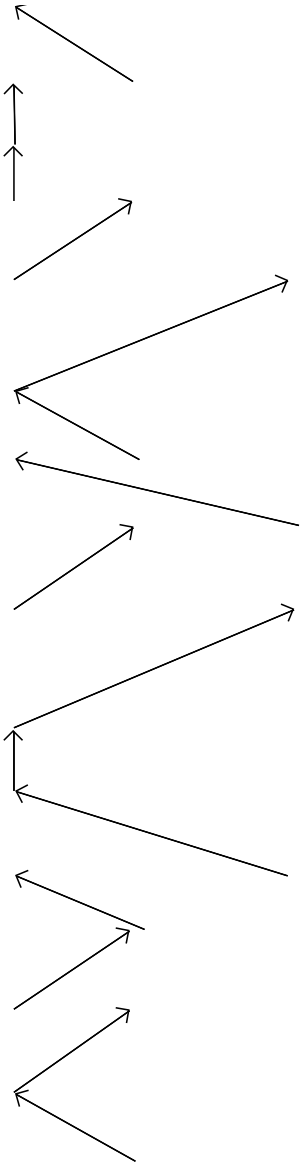
h_1

h_0

It would help if the pre-activations at each layer were unit gaussians





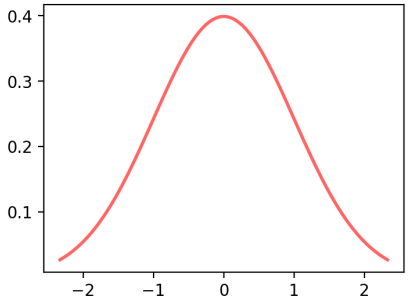


h_4

h_3

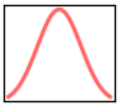
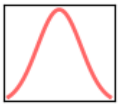
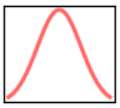
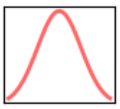
h_2

h_1



Why not explicitly ensure this by standardizing the pre-activation ?

$$\hat{s}_{ik} = \frac{s_{ik} - E[s_{ik}]}{\sqrt{var(s_{ik})}}$$

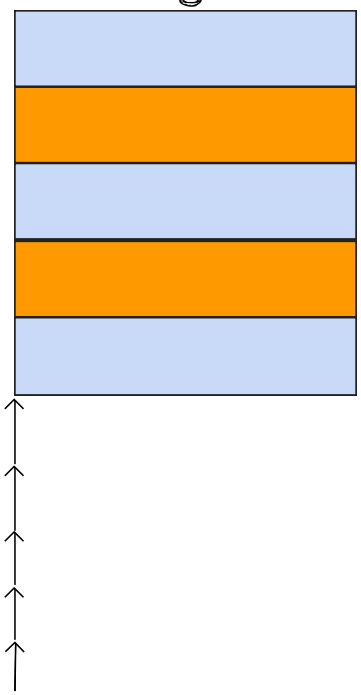
But how do we compute $E[s_{ik}])$ and $Var[s_{ik}]$?

We compute it from a mini-batch

Thus we are explicitly ensuring that the distribution of the inputs at different layers does not change across batches

This is what the deep network will look like with Batch Normalization

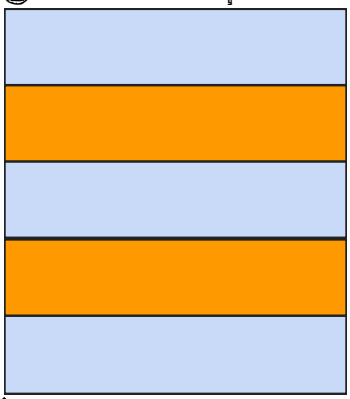
Is this legal ?



Yes, it is because just as the *tanh* layer is differentiable, the Batch Normalization layer is also differentiable

Hence we can backpropagate through this layer

Catch: Do we necessarily want to force a unit gaussian input to the *tanh* layer?



tanh
tanh
tanh
BN
BN

Why not let the network learn what is best for it?
After the Batch Normalization step add the following step:

$$y^{(k)} = \gamma^k \hat{s}_{ik} + \beta^{(k)}$$

What happens if the network learns:

$$\gamma^k = \sqrt{var(x^k)}$$

$$\beta^k = E[(x^k)]$$

$\gamma^k$and β^k are additional parameters of the network.
We will recover s_{ik}

In other words by adjusting these additional parameters the network can learn to recover

s_{ik} if that is more favourable

x_1^2
x_2^2
x_3^2
x_4^2
x_5^2

x_1^1
x_2^1
x_3^1
x_4^1
x_5^1

x_1^m
x_2^m
x_3^m
x_4^m
x_5^m

. . .

Accumulator

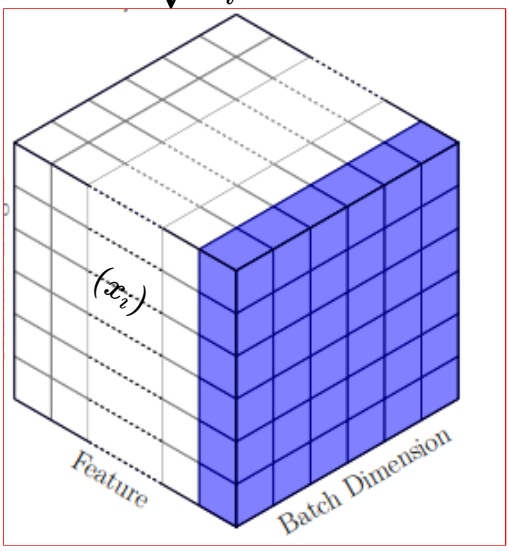
Accumulated activations for m training samples

Let x_i^j denotes the activation of i^{th} neuron for j^{th} training sample

$$\mu_{i=2} = \frac{1}{m}\sum_{j=1}^{m} x_2^j$$

$$\sigma_i^2 = \frac{1}{m}\sum_{j=1}^{m}(x_2^j - \mu_i)^2$$

$$\hat{x_i} = \frac{x_i - \mu_i}{\sqrt{\sigma_i^2 + \epsilon}}$$



Source

l

We have three variables l, i, j involved in the statistics computation. Let's visualize these as three axes that form a cube.

Let us associate an accumulator with l^{th} layer that stores the activations of batch inputs.

$$\hat{y_i} = \gamma\hat{x_i} + \beta$$

x_1^2
x_2^2
x_3^2
x_4^2
x_5^2
x_1^1
x_2^1
x_3^1
x_4^1
x_5^1
x_1^m
x_2^m
x_3^m
x_4^m
x_5^m
$\dots$

$Accumulator$

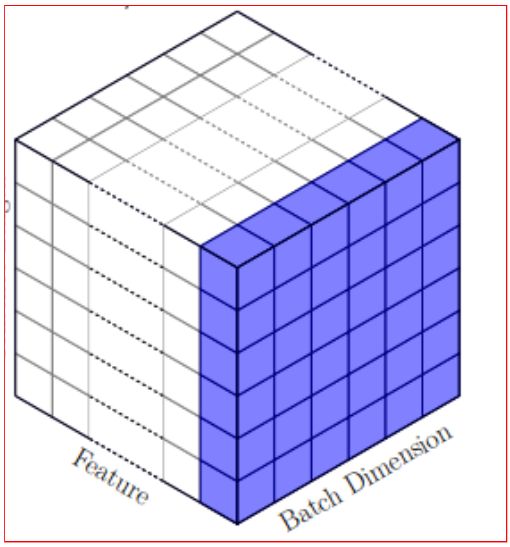
Accumulated activations for m training samples

Let x_i^j denotes the activation of i^{th} neuron for j^{th} training sample

$$\mu_{i=3} = \frac{1}{m}\sum_{j=1}^{m} x_3^j$$

$$\sigma_i^2 = \frac{1}{m}\sum_{j=1}^{m}(x_3^j - \mu_i)^2$$

$$\hat{x_i} = \frac{x_i - \mu_i}{\sqrt{\sigma_i^2 + \epsilon}}$$

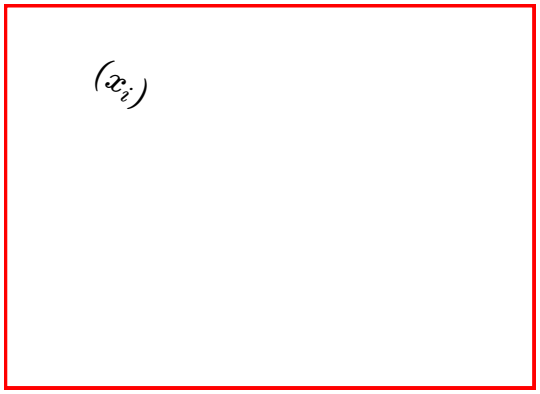


Source

l

We have three variables l, i, j involved in the statistics computation. Let's visualize these as three axes that form a cube.

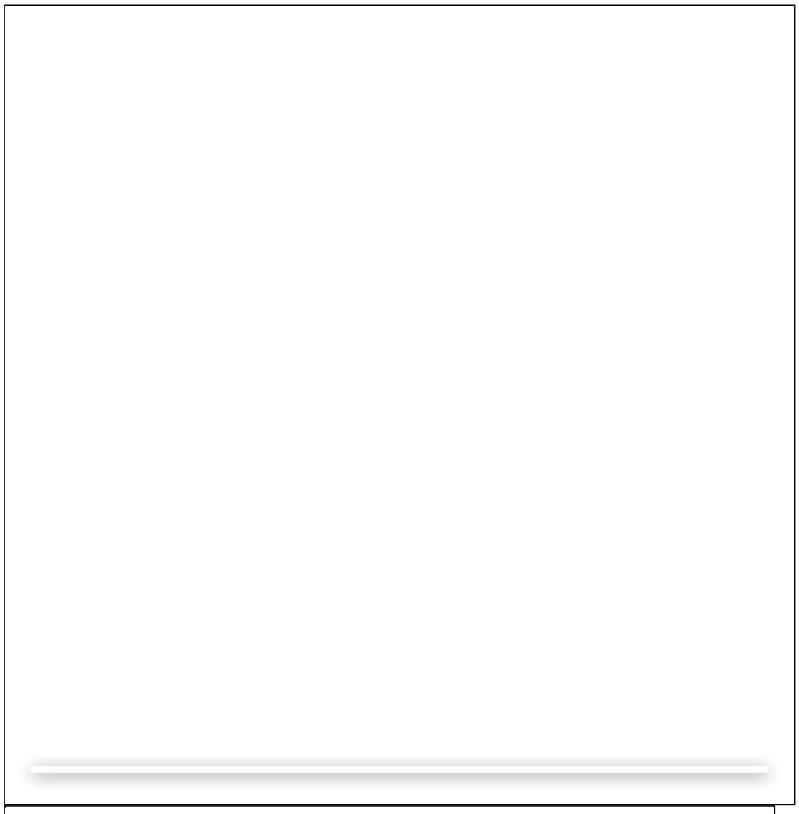
Let us associate an accumulator with l^{th} layer that stores the activations of batch inputs.

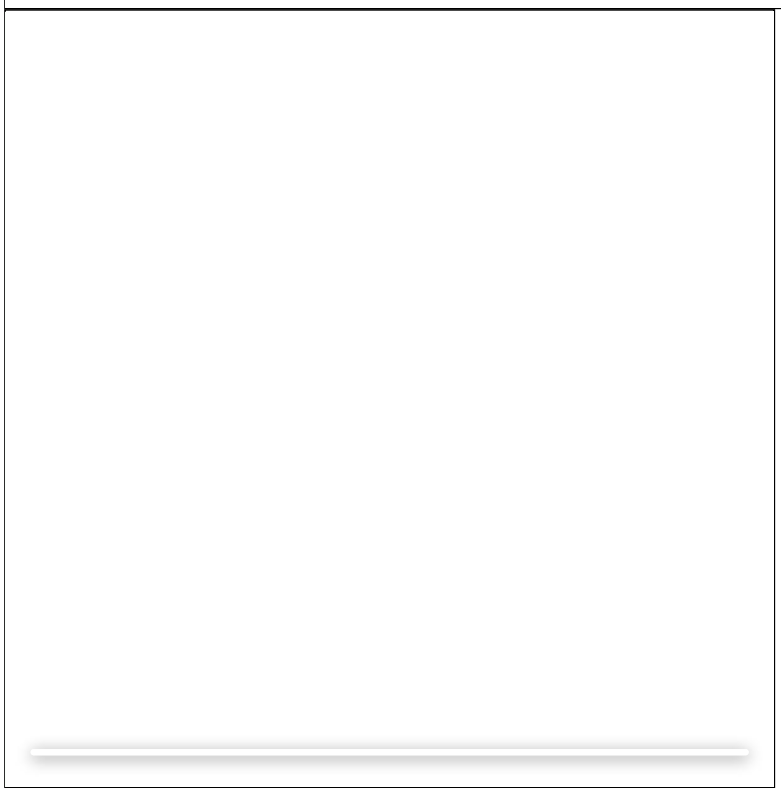


$$\hat{y}_i = \gamma \hat{x}_i + \beta$$

j

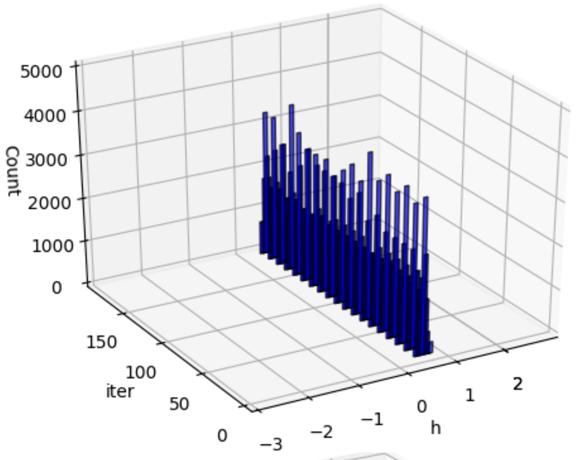
We will now compare the performance with and without ba
normalization on MNIST data using 7 layers....

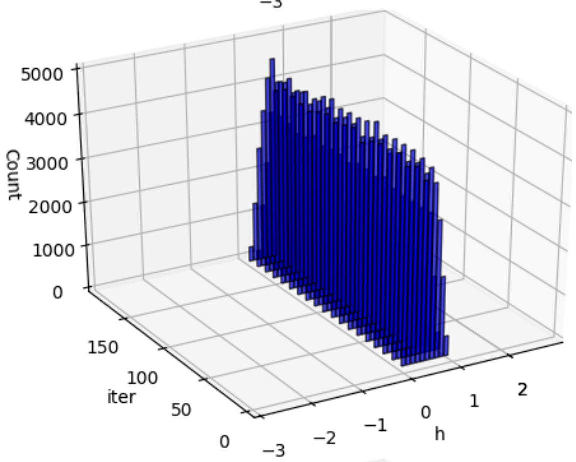


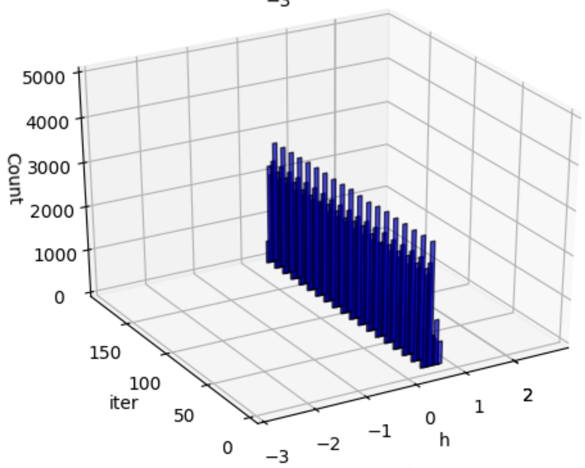


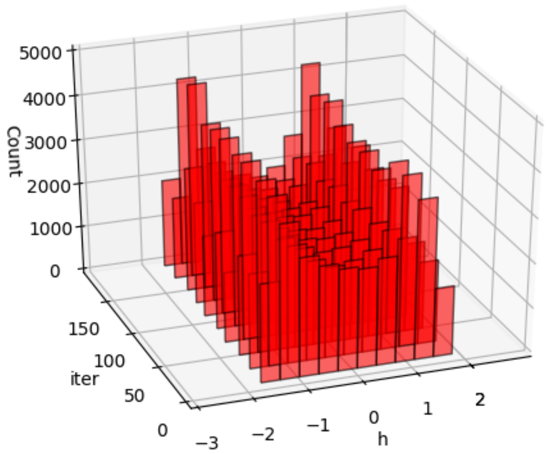
Without Batch Normalization

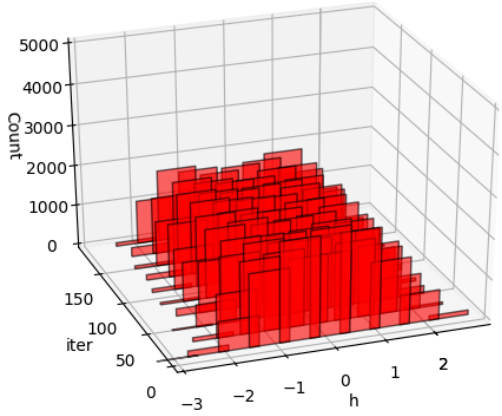
With Batch Normalization

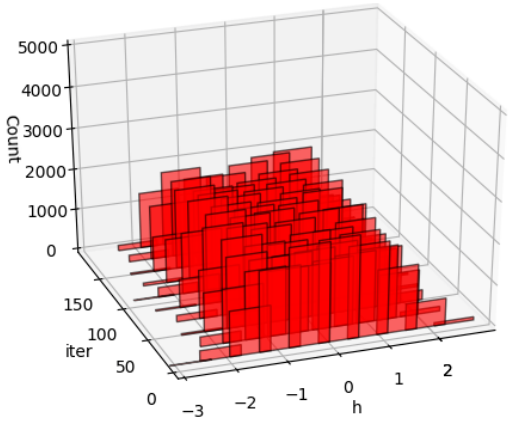

5000
4000
3000
2000
1000
0
Count
150
100
iter
50
0
−3
−2
−1
0
1
2
h


5000
4000
3000
2000
1000
0
Count
150
100
iter
50
0
−3
−2
−1
0
1
2
h


5000
4000
3000
2000
1000
0
Count
150
100
iter
50
0
−3
−2
−1
0
1
2
h


5000
4000
3000
2000
1000
0
Count
150
100
iter
50
0
−3
−2
−1
0
1
2
h





h_1

h_4

h_7

Blue: Without BN
Red: With BN

Sigmoid Activation, w=0.1*np.random.randn()

Limitations of Batch Normalization

The accuracy of estimation of mean and variance depends on the size of m. So using , due to memory constraint for data like image and videos, a smaller size of m results in high error.

Because of this, we can't use a batch size of 1 at all (i.e, it won't make any difference, $\mu_i = x_i, \sigma_i = 0$)

Many alternatives were proposed like Group normalization, Instance Normalization which are domain specific. However, the simplest one among them is called **Layer Normalization.**

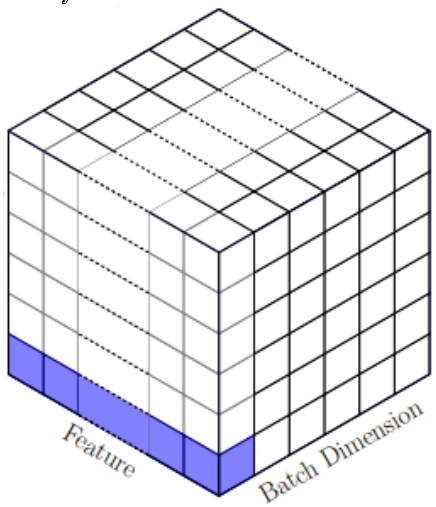
Other than this limitation, it was also empirically found that the naive use of BN leads to performance degradation in NLP tasks (**source**).

There was also a **systematic study** that validated the statement and proposed a new normalization technique (by modifying BN) called powerNorm.

$$\mu_i = \frac{1}{m}\sum_{j=1}^{m} x_i^j$$

$$\sigma_i^2 = \frac{1}{m}\sum_{j=1}^{m}(x_i^j - \mu_i)^2$$

Layer Normalization



The computation is simple. Take the average across outputs of hidden units in the layer. Therefore, the normalization is independent of samples in a batch. This allows us to work with a batch size of 1 (if needed as in the case of RNN)

x_1

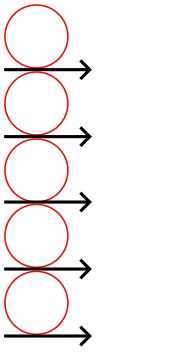
x_2

x_3

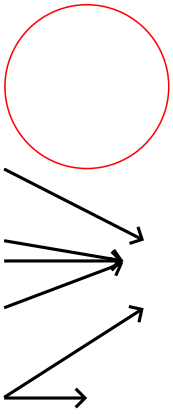
x_4

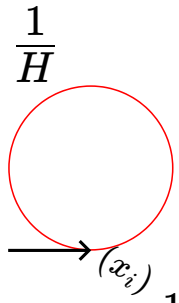
x_5

l^{th} layer



$\sum$





$$\mu_l = \frac{1}{H}\sum_{i=1}^{H} x_i$$

$$\sigma_l = \sqrt{\frac{1}{H}\sum_{i=1}^{H}(x_i - \mu_l)^2}$$

l

$$\hat{x}_i = \frac{x_i - \mu_l}{\sqrt{\sigma_l^2 + \epsilon}}, \quad \forall i$$

$$\hat{y}_i = \gamma \hat{x}_i + \beta$$

Speaker notes